Highway to Renewable Energy Systems

H2RES v1.0 Report (Draft)

User Manual for Research, Scenario Design, and Model Deployment

Document Type: User Manual

Reviewed by: .....

Date: June 3, 2026

Contents

1 Purpose and Scope

This manual is intended to support use, adaptation, and extension of H2RES model in research and applied energy-system analysis. It complements the Mathematical Formulation Report.

This document focuses on:

- End-to-end user workflow from scenario design to result interpretation.
- Input data and cross-file consistency.
- Practical traceability between input files, internal parameters, and model blocks.
- Troubleshooting patterns observed from the current repository structure.
- Research-grade reproducibility practices for future case studies.

1.1 What H2RES Solves

H2RES model is an hourly energy system optimization model that co-optimizes:

- Electricity generation and capacity investment.
- Intertemporal storage (hydro, stationary electrochemical storage, EV fleet storage).
- Sector coupling with heat, cooling, and hydrogen pathways.
- Policy instruments such as RPS trajectories, carbon limit, and optional reserve modules.

1.2 Intended Users

- **New users:** run the baseline Croatia case and understand outputs.
- **Researchers:** prepare new case-study datasets and scenario assumptions.
- **Developers:** modify equations after validating data and workflow consistency.

1.3 How to Use This Manual

Reading objective	Start here	What this part gives you
Understand what H2RES represents	Sections 1 and 6	System scope, carrier interactions, and the physical meaning of the main subsystems before touching input files or code.
Run the model for the first time	Sections 2 and 3	Execution order, first-run checks, and the minimum evidence that a baseline run completed correctly.
Modify scenario assumptions	Sections 4 and 5	Which settings belong in `main.py`, which files control data, and how cross-file consistency must be preserved.
Adapt the model to a new case study	Sections 7 and 8	What to modify, in what order, and how those changes propagate into model blocks and internal parameters.
Interpret and validate results	Sections 9 to 12	Output reading order, troubleshooting, validation logic, and reproducibility expectations.

1.4 Core Abbreviations and Nomenclature

This table consolidates the abbreviations that are used most often in the user manual and in the Mathematical Formulation Report.

Abbreviation	Expanded term	Practical meaning in H2RES
CEEP	Critical excess / curtailed electric-energy	Variable used to represent the excess electrical energy production to total demand.
CHP	Combined heat and power	Units that jointly produce electricity and useful heat under coupling constraints.
COP	Coefficient of performance	Heat-pump conversion ratio from electricity input to heat or cooling output.
DH	District heating	Aggregated thermal markets linked to CHP units, heat pumps, and boilers.
EV	Electric vehicle	Aggregate fleet-storage representation used for transport flexibility.
FC	Fuel cell	Hydrogen-to-electricity conversion technology.
H2	Hydrogen	Energy carrier modeled through electrolysis, storage, direct demand, and fuel-cell use.
HDAM	Hydro dam reservoir	Reservoir hydro unit with intertemporal storage dynamics.
HP	Heat pump	Electricity-driven heat or cooling technology in DH or general-demand blocks.
HPHS	Pumped-hydro storage	Hydro storage technology with intertemporal water inventory behavior.
HROR	Hydro run-of-river	Non-dispatchable hydro technology without reservoir optimization.
MILP/LP	Mixed-integer / linear program	Optimization model class used depending on active modules and formulation choices.
NPV	Net present value factor	Intertemporal discounting weight applied in the objective.
NTC	Net transfer capacity	Import-capacity ceiling used in electricity-balance logic.
RPS	Renewable portfolio standard	Policy block enforcing a minimum renewable generation share.
SOC	State of charge	Intertemporal inventory state for EV, battery, hydro, heat, or hydrogen storage.

1.5 Index Symbols Used Throughout the Manual

The mathematical and traceability sections rely on a compact set of recurring indices. Listing them in one place reduces the need to cross-check the Mathematical Formulation Report while reading the manual.

Index	Typical set or object	Meaning in H2RES
u	units	Electricity-generation unit in the main plant registry.
t	periods	Hourly or time-slice period within a model year.
y	years	Model year in the planning horizon.
b	storage or boiler subsets	Stationary battery unit or, depending on the block, thermal boiler index.
h	HeatPump_units	Heat-pump technology or market-linked heat-pump identifier.
m	CHP or DH-market subsets	CHP-linked heat market or district-heating market identifier.
s	demand or storage subsets	Demand sector or hydrogen-storage index, depending on the equation block.
e	Elec_h2_units	Electrolyzer identifier.
c	FC_units	Fuel-cell identifier.
f	fuel subsets	Fuel type, especially in industry or emissions blocks.
q	hydrogen-demand subsets	Hydrogen-demand sector or use category.

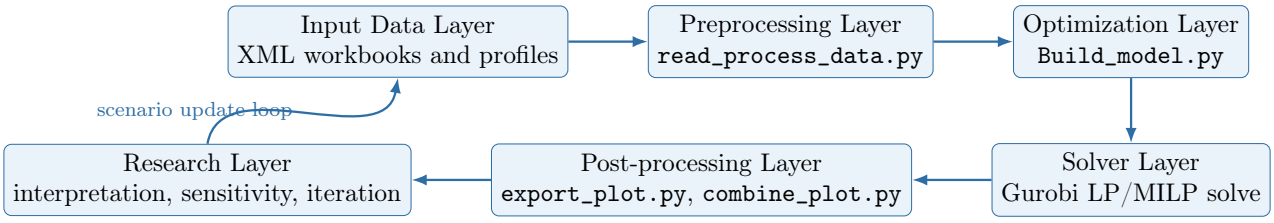

Figure 1: Layered architecture of H2RES from data engineering to research decisions.

2 Recommended End-to-End Workflow

This section is operational. It describes the sequence a user follows to run H2RES from repository root to interpreted results.

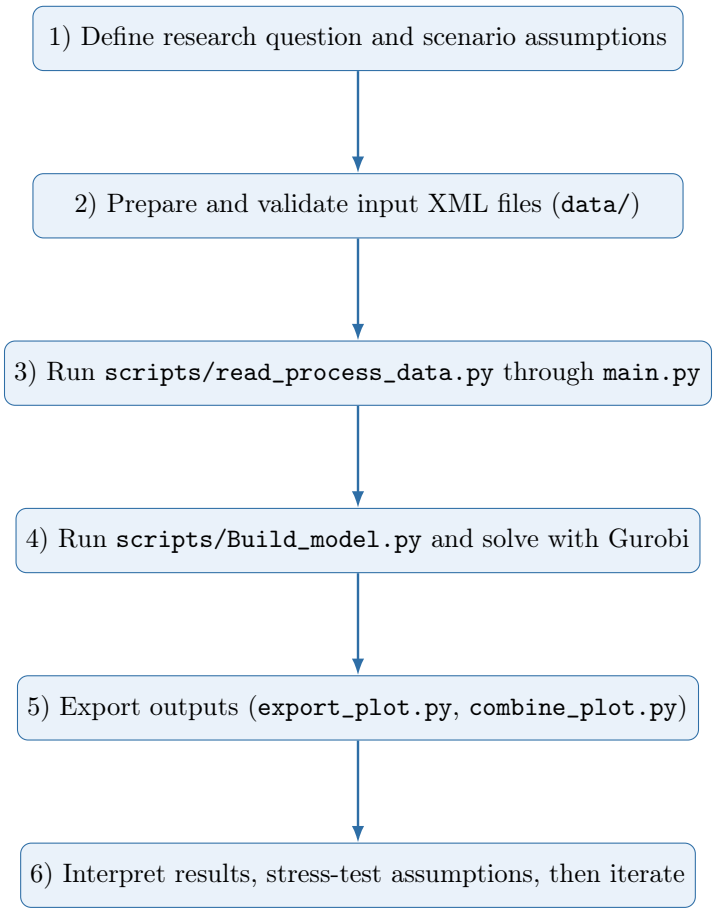

Figure 2: User workflow aligned with repository execution order.

2.1 Operational Sequence in this Repository

The current `main.py` executes:

1. `scripts/read_process_data.py`
2. `scripts/Build_model.py`
3. `scripts/export_plot.py`
4. `scripts/combine_plot.py`

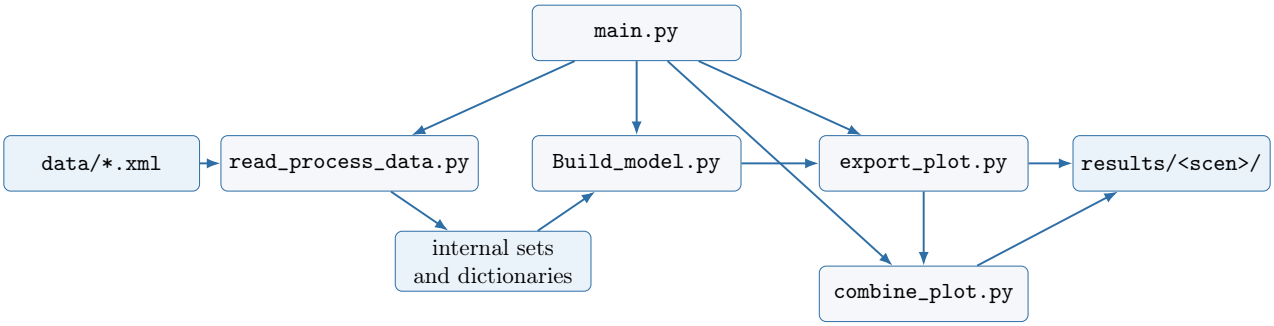

Figure 3: Execution dependency map of repository scripts and artifacts.

2.2 Execution Command

python main.py or uses open-source Integrated Development Environment such as visual studio code/spyder

2.3 Runtime Notes

- The run should be launched from repository root so all relative paths resolve.
- `export_plot.py` removes and recreates `./results/<scenario>/`; archive old runs before reusing a scenario name.
- The scenario name should be changed for each model run to avoid losing information.

3 First Baseline Run

For a new user, the first target should be a clean baseline run with no equation changes and no schema changes in the input files. The purpose of this first pass is to confirm that the environment, file paths, active scenario switches, and export logic are behaving as expected before any scientific interpretation begins.

3.1 Pre-launch baseline checklist

- Confirm that the run is launched from repository root so all relative paths resolve correctly.
- Open `main.py` and changes the active scenario name `scen`, key boolean switches, and the main policy vectors.
- Confirm that all mandatory input files referenced in Section 5 are present in the expected data folder.
- Archive **any old folder with the same scenario name**, because `export_plot.py` recreates the result directory.
- Do not modify equations, identifiers, or workbook schemas before the baseline run succeeds once.

3.2 Baseline execution procedure

1. Open `main.py` and review only the scenario-level assumptions that you intend to use for the first run.
2. Run `python main.py` from repository root or uses open-source Integrated Development Environment such as visual studio code/spyder.
3. Allow the preprocessing stage to build internal sets and dictionaries before the optimization model is instantiated.
4. Check solver-status output after model solution.
5. Confirm that **a new folder** is created under `results/<scenario_name>/`.
6. Inspect the first-pass outputs listed in Section 9 before making any further scenario changes.

3.3 What indicates a successful first run

Checkpoint	Expected sign	Why it matters
Preprocessing stage	No missing-key or schema error during data loading	Confirms that identifiers and time axes are consistent enough to build model dictionaries.
Optimization stage	Solver reports a feasible/optimal completion status used by the repository workflow	Confirms that the data package and scenario assumptions are numerically consistent.
Results folder	`results/<scenario_name>/` is recreated and populated	Confirms that post-processing scripts ran and that outputs were exported to the expected location.
Summary file	`report.txt` exists	Gives a first high-level check of run completion and headline metrics.
Core tables	At least the main generation, storage, and sector-coupling artifacts are written	Confirms that the main model blocks were active and exported correctly.

Solver-status note: the labels `Status==2` and `Status==13` are Gurobi solver status codes, not H2RES-specific completion labels. For a first baseline run, `Status==2` is the preferred outcome because it indicates an optimal solution. `Status==13` indicates that Gurobi returned a feasible but suboptimal solution, typically after reaching a stopping condition such as a time or numerical limit. If `Status==13` appears, inspect the solver log before drawing scientific conclusions.

3.4 Common first-run discipline

For the first successful run, change as little as possible. If the baseline fails, do not move immediately to new policies, new technologies, or new datasets. First establish whether the issue comes from **environment setup, file matching, time indexing, or scenario-switch logic**.

4 Scenario Design in `main.py`

4.1 Boolean Switches and Direct Effects

Parameter	Type	Default	User-level meaning	Effect on model behavior
`rps_inv`	bool	True	Activate renewable share policy.	If False, RPS minimum share is not enforced.
`carbonLimit`	bool	True	Activate annual CO2 cap policy.	If False, CO2 cap constraints are removed.
`res_inv`	bool	True	Allow renewable/flex investment pathways.	If False, new RES and FC investment is forced to zero in code.
`hydro_storage`	bool	True	Activate hydro storage dynamics.	If False, HDAM/HPHS intertemporal constraints are skipped.
`exports_dat`	bool	False	Use export-aware power balance variant.	If False, standard domestic balance formulation is used.
`save_csv`	bool	True	Save output tables.	If False, most XLSX outputs are not written.
`ceep_limit`	bool	True	Enforce annual CEEP share bound.	If False, annual CEEP bound is disabled.
`NoResToHeatInv`	bool	False	Disable power-to-heat route.	If True, forces `heat_pump_cap=0`, `ResToHeat=0`, `heat_pump_out=0`.
`PrimaryReserve`	bool	False	Activate primary reserve module.	If False, primary reserve variables/constraints are excluded.
`SecondaryReserve`	bool	False	Activate secondary reserve module.	If False, secondary reserve variables/constraints are excluded.

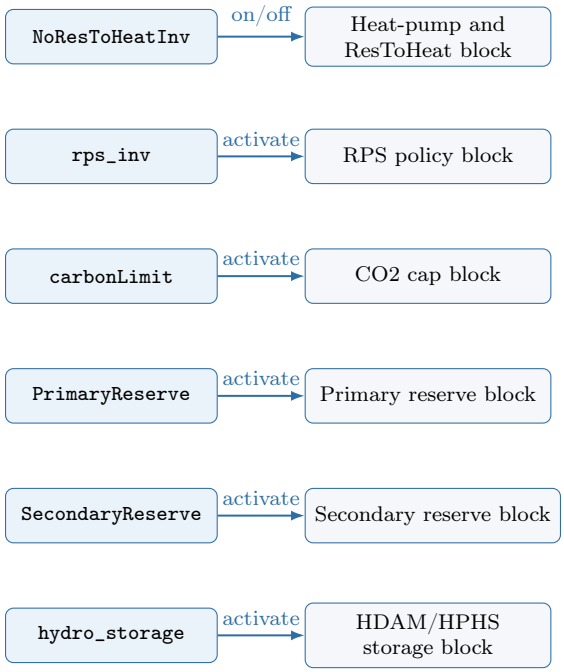

Figure 4: Scenario-switch activation map linking booleans to model submodules.

4.2 Recommended Switch Combinations for First Studies

Boolean switches are easier to use when treated as scenario recipes rather than as isolated flags. The combinations below are not exhaustive, but they provide a practical activation order for new users working with an integrated-sector model.

Study objective	Recommended switch combination	Why this is a good starting point
Baseline familiarization	Keep default policy switches, keep `NoResToHeatInv=False`, leave `PrimaryReserve=False` and `SecondaryReserve=False`.	Preserves the integrated electricity-heat-hydrogen structure while avoiding reserve complexity during the first run.
Debugging infeasibility or data issues	Temporarily relax `rps_inv=False`, `carbonLimit=False`, `ceep_limit=False`; keep reserve switches off.	Helps distinguish data or indexing failures from policy-driven infeasibility.
Sector-coupling exploration	Keep the baseline recipe and vary heat/H2 demand or technology inputs before turning on reserves.	Makes it easier to interpret heat-pump, hydrogen, and industrial-switching outputs without reserve-side interactions.
Reserve-focused study	Start from a validated baseline, then activate `PrimaryReserve=True`; activate `SecondaryReserve=True` only after the primary-reserve run behaves plausibly.	Reserve modules add additional eligibility and feasibility constraints, so staged activation isolates failures more cleanly.
Policy stress test	Keep all data fixed, then tighten `rps`, `CO2_limit`, or carbon-price trajectories one at a time.	Preserves causal interpretation by changing policy intensity without simultaneously changing data or module topology.

Practical rule: when several modules are new to the user, activate complexity in layers: baseline first, then heat/hydrogen interpretation, then reserves, and only after that stronger policy stress tests.

4.3 Policy and Economic Trajectories

Parameter	Typical structure	Practical interpretation	Modeling effect
`rps`	list indexed by year	Renewable share target path.	Higher values tighten renewable generation requirements.
`CO2_limit`	list indexed by year	Annual emissions cap path.	Lower values tighten decarbonization feasibility.
`carbon_price`	list indexed by year	CO2 marginal penalty path.	Penalizes carbon-intensive dispatch progressively.
`ceep_parameter`	scalar	Max annual CEEP fraction of annual electric needs.	Lower value reduces tolerated adequacy slack.
`NPV`	list indexed by year	Discount factors for intertemporal objective weighting.	Lower future factors reduce weight of long-term costs.
`Import_Price`, `Imp_price_inc`	scalar(s)	Import cost level and growth.	Shifts the competitiveness of imports relative to domestic dispatch and investment.

Currency conversion note: Throughout this user manual, all monetary inputs are interpreted as EUR-based scenario assumptions.

4.4 Transport and Flexibility Assumptions

The transport-flexibility block in `main.py` defines an aggregated EV fleet rather than individual vehicles. The user specifies a small set of direct assumptions, and the script converts them into annual charge-power and storage-energy envelopes that later appear in the EV state-of-charge recursion and electricity-balance constraints. The table below is intentionally scenario-agnostic: current numeric values belong to the active `main.py` configuration, while the manual focuses on interpretation and model role.

Parameter in `main.py`	Unit	Meaning in the model
`V2G_cost`	EUR/MWh$_e$	Marginal cost or penalty assigned to EV-to-grid flexibility use in the objective.
`ev_Demand_year`	p.u.	Annual multiplier applied to the EV transport-load requirement, allowing year-specific scaling of the transport-energy withdrawal represented in the EV balance.
`ev_sto_min`	p.u.	Minimum allowed fraction of aggregate EV state of charge.
`ev_Grid_eff`	p.u.	Charging/discharging efficiency used in the EV storage recursion.
`number_of_veh`	vehicles	Total EV fleet represented by the aggregate storage model.
`average_ch_rate`	kW/vehicle	Per-vehicle charging-power assumption used to derive fleet charging capacity.
`average_bat`	kWh/vehicle	Per-vehicle battery-energy assumption used to derive fleet storage capacity.

Two quantities are then derived directly in `main.py`:

$$\texttt{charging\_veh} = \texttt{number\_of\_veh} \cdot \texttt{average\_ch\_rate}/1000,$$
$$\texttt{stor\_veh} = \texttt{number\_of\_veh} \cdot \texttt{average\_bat}/1000.$$

These aggregate fleet totals are not used directly as decision variables. Instead, they are converted into year-specific bounds:

Derived quantity	Unit	Meaning in the model
`charging_veh`	MW_e	Aggregate fleet charging-power potential derived from fleet size and per-vehicle charging rate. It is an intermediate quantity used to construct yearly EV charging/discharging limits.
`stor_veh`	MWh_e	Aggregate fleet battery-energy potential derived from fleet size and per-vehicle battery size. It is an intermediate quantity used to construct yearly EV storage limits.
`ev_Grid_P`	MW_e	Annual EV grid-interface power limit, later mapped to $\texttt{ev\_Grid\_P\_max}_y$. It caps hourly charging and discharging rates of the aggregated EV fleet.
`ev_stor`	MWh_e	Annual aggregate EV storage-energy limit, later mapped to $\texttt{ev\_sto\_max}_y$. It caps the EV state of charge in each model year.

In practical terms, the scenario designer specifies the total theoretical fleet size and technical characteristics first, and then uses year-dependent EV bounds to decide how much of that theoretical flexibility is actually accessible to the power system in each model year. Heat and hydrogen flexibility are less directly controlled from `main.py`. Those modules depend primarily on technology and demand records loaded from:

- `flex_tech_HR_2020_2050_sdewes.xml`,
- `heat_demand_HR_2020_2050_sdewes.xml`,
- `cooling_demand_HR_2020_2050_sdewes.xml`, and
- `demand_H2_2020_2050_sdewes.xml`.

4.5 User-Defined Intertemporal Trajectories in `main.py`

Besides policy vectors, `main.py` defines several year-indexed trajectories that shape technology evolution and the persistence of pre-existing assets. These vectors must have the same length and ordering as the model year.

Parameter in `main.py`	Unit	Interpretation and modeling role
`TechChangeSolar`	p.u.	User-defined year path for solar technology-change weighting. It is mapped into $\texttt{TechChange}_{u,y}$ for solar units and changes the relative intertemporal attractiveness of investment in the objective.
`TechChangeWind`	p.u.	Same logic as above for wind units. Lower future values imply a progressively more favorable investment environment relative to the initial year.
`ThermalDecomInd`	p.u.	Remaining surviving-share profile for pre-existing industrial thermal-boiler capacity. It controls how much inherited industrial thermal capacity remains available over time.
`HeatPumpDecomInd`	p.u.	Remaining surviving-share profile for pre-existing industrial heat-pump capacity. It determines how much of the initial stock survives before new investment is needed to maintain service.
`ThermalDecomDH`	p.u.	Remaining surviving-share profile for district-heating thermal-boiler capacity inherited from the base year.
`StaStoDecomInd`	p.u.	Remaining surviving-share profile for pre-existing stationary battery energy capacity. It enters the upper bound on battery SOC through the surviving initial stock term.

The key distinction is the following:

- `TechChangeSolar` and `TechChangeWind` are forward-looking economic modifiers associated with future investment conditions.
- `ThermalDecomInd`, `HeatPumpDecomInd`, `ThermalDecomDH`, and `StaStoDecomInd` are surviving-share trajectories for capacity that already exists at the beginning of the horizon.

For scenario design, this means that a user can modify or relax long-term transition pressure in two different ways: by changing the economic attractiveness of new technologies, or by accelerating the retirement of inherited assets.

5 Input Data Engineering Guide

5.1 Why this is critical

For most new users, input preparation is the dominant workload. The model relies on joins across files using names and time indices; any mismatch can break preprocessing or invalidate economics.

5.2 Required Input Files and Roles

Input file	Primary role	Expected key fields	Main internal outputs	Critical consistency check
genco_data_HR_sdewes.xml	Core generation and technical master table	unit_name, fuel_type, technology, cap_mw, ramp, CO2, CHP fields	sets (units, disp_units, etc.), max_load, cap_factor, CO2_factor, reserve factors	unit_name must align with inflow/availability columns and CHP demand mapping
demand_2020_2050_sdewes.xml	Electricity demand by sector/hour/year	year, period, sector columns	periods, years, demand	complete coverage of all (t, y) used in model
fuel_cost_2020_2050_sdewes.xml	Time-dependent fuel prices	year, period, fuel columns	var_cost, heat_var_cost, industry_fuels_price	fuel labels must match genco, thermal, and industry fuel names
ncre_aval_factor_HR_2020_2050_sdewes.xml	Availability factors for non-dispatch units	year, period, one column per unit	aval_factor_plant	unit columns must match units used in non-dispatch equations
scaled_inflows_HR_2020_2050_sdewes.xml	Hydro inflow profiles (HDAM/HPHS)	normalized header with year, period, hydro unit columns	hdam_inflow, hphs_inflow	inflow columns must match HDAM/HPHS unit names in genco data
import_export_HR_2020_2050_sdewes.xml	Import/export capacity trajectories	year, period, Import_capacity, total_exports	import_ntc, exports	period index must align with demand periods
heat_demand_HR_2020_2050_sdewes.xml	Heat demand for CHP markets + aggregate sectors	year, period, CHP market columns, general_demand, Industry_HT	heat_demand, Industry_HT_demand	CHP market names must align with DH mapping and CHP units
cooling_demand_HR_2020_2050_sdewes.xml	Cooling demand by market	year, period, market columns, general_cooling	cooling_demand	market names must align with HP DH structure
ev_transp_load.xml	EV availability and transport energy profile	year, period, ev_availability, ev_transp_load	ev_availability, ev_transp_load	all model periods per year must exist; no missing hours
demand_H2_2020_2050_sdewes.xml	Hydrogen demand by sector/hour/year	year, period, sector columns	H2_demand	sector labels should match expected sector set logic
flex_tech_HR_2020_2050_sdewes.xml	Flex assets, costs, COP, industry shares, DH mapping	sheets: flex_tech, thermal_tech, DistrictHeating, tech_cost_years, Industry, COP_HP	FC_*, HP_*, H2_*, Sta_*, HP_COP_period, fuel_share_max	sheet names and technology labels must stay synchronized across tabs

5.3 Interpreting the genco master table

The `genco_data_HR_sdewes.xml` file is the master unit table of the electricity-supply side in this repository. Its columns define the technical identity, cost structure, operational flexibility, reserve eligibility, storage behavior, CHP coupling, and long-term evolution of each generation asset represented in H2RES. In practical terms, this file acts as the central asset registry from which the preprocessing step builds unit subsets, cost parameters, emissions factors, operational limits, and intertemporal investment or retirement logic. Some fields describe short-term dispatch behavior, while others describe long-term capacity evolution under the scenario horizon.

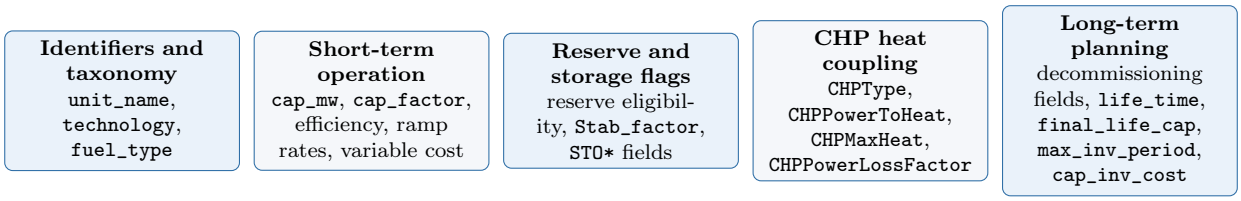

Figure 5: The file mixes identifiers, operational data, storage/reserve fields, CHP coupling terms, and planning trajectories.

Local genco field	Unit	Meaning	Main role in H2RES
`unit_name`	-	Unique identifier of the generating unit.	Forms the master key used for model sets and for matching with time-series columns in availability, inflow, and some demand-linked files.
`cap_mw`	MW	Installed nominal power capacity.	Used to define $\texttt{max\_load}_u$, generation limits, hydro conversion scales, and initial installed capacity.
`fuel_type`	-	Primary fuel classification of the unit.	Drives fuel-cost assignment, fuel-based subsets, and emissions accounting.
`decom_start_existing_c`	year index	First modeled year at which the inherited capacity starts its retirement trajectory.	Used to build surviving existing-capacity terms for legacy assets.
`life_time`	year index	Last year used in the new-capacity survival calculation.	Combined with `decom_start_new` and `final_life_cap` to derive a geometric decommissioning rate for new builds.
`decom_start_new`	year index	First modeled year at which newly installed capacity begins its decommissioning trajectory.	Defines when new investments start to decay in the intertemporal capacity equations.
`final_life_cap`	p.u.	Residual fraction of capacity remaining at the end of the lifetime window.	Used in preprocessing to compute the annualized decommissioning rate of new capacity.
`max_inv_period`	MW	Maximum new capacity addition allowed in one modeled period.	Caps annual investment decisions for each unit.
`cap_factor`	p.u.	Capacity or utilization factor used in upper-bound logic.	Scales effective generation envelopes and annual production potential.
`efficiency`	p.u.	Conversion efficiency of the unit.	Used in variable-cost calculations and, for hydro/storage technologies, as discharge efficiency in storage dynamics.
`cost_no_fuel`	EUR/MWh or equivalent	Non-fuel variable operating cost term.	Contributes to electricity-generation variable cost after fuel-price merging.
`cap_inv_cost`	EUR/MW	Capital cost of new investment.	Enters the capacity-expansion objective term.
`RampingCost`	EUR/MW	Cost coefficient for inter-hour output changes.	Used in ramping-cost linearization.
`CO2Intensity`	tCO2/MWh	Direct carbon intensity of generation.	Used for CO2 accounting and carbon-cost terms.
`technology`	-	Technology label of the unit, such as `HDAM`, `HPHS`, `HROR`, `PHOT`, or thermal types.	Defines operational subsets and determines which constraint blocks apply to each unit.
`RampUpRate`	p.u./time step	Maximum upward change in output.	Used in dispatch ramp-up constraints.

Local genco field	Unit	Meaning	Main role in H2RES
`RampDownRate`	p.u./time step	Maximum downward change in output.	Used in dispatch ramp-down constraints.
`PrimaryReserve`	flag	Eligibility marker for primary reserve provision.	Determines whether a unit enters the primary-reserve eligible set.
`SecondaryReserve`	flag	Eligibility marker for secondary reserve provision.	Determines whether a unit enters the secondary-reserve eligible set.
`Stab_factor`	p.u.	Stability or reserve-scaling factor for reserve feasibility.	Used in reserve-down feasibility relationships.
`CHPType`	flag/category	CHP activation/type indicator in the local schema.	Determines whether CHP-specific coupling and heat-storage parameters are read for the unit.
`STOCapacity`	MWh or MWh_{th}	Storage capacity associated with hydro or CHP-linked storage.	Used as hydro reservoir energy limit or CHP thermal storage capacity, depending on the unit type.
`STOSelfDischarge`	p.u./day	Storage self-discharge or daily heat-loss rate.	Used for CHP thermal-storage losses and interpreted consistently with storage-type logic.
`STOMaxChargingPower`	MW	Maximum charging power for storage-capable units.	Defines the upper charging limit for technologies that can absorb electricity into storage.
`STOChargingEfficiency`	p.u.	Charging efficiency for storage-capable units.	Determines how much stored energy is retained from each unit of charging input.
`CHPPowerToHeat`	p.u.	Power-to-heat coupling ratio of CHP operation.	Converts extracted electric-equivalent input into useful heat in CHP constraints.
`CHPPowerLossFactor`	p.u.	Electricity-output loss associated with heat extraction.	Reduces feasible electric generation when CHP heat is produced.
`CHPMaxHeat`	MW_{th}	Maximum heat-production capability of the CHP unit.	Bounds CHP heat output in coupled heat-power operation.

Note: the `genco` file mixes short-term operational parameters with long-term planning parameters. Fields such as `decom_start_existing_cap`, `decom_start_new`, `life_time`, `final_life_cap`, and `max_inv_period` are not dispatch inputs; they are scenario-planning parameters used by the local preprocessing and investment model.

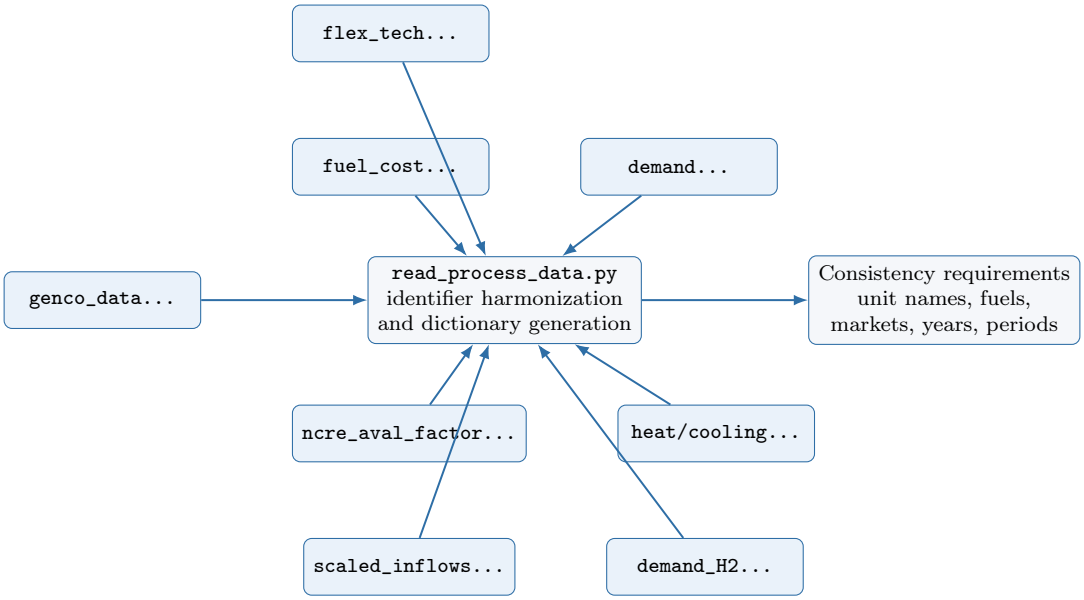

Figure 6: Cross-file consistency network required before model construction.

5.4 How Wide-Format Files Become Long-Format Tables

Several files arrive in a wide format, where each technology or unit appears as a separate column. During preprocessing, those files are reshaped into long-format tables keyed by `year`, `period`, and

an identifier such as `unit_name` or fuel label. For new users, this transformation is one of the most important ideas behind the matching rules presented in this section.

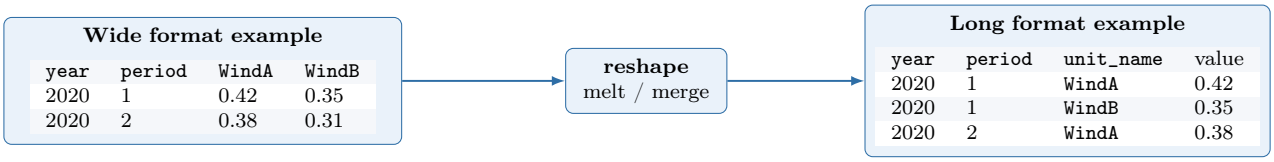

Figure 7: Conceptual wide-to-long transformation used for files such as renewable availability and hydro inflows. Matching then occurs on `year`, `period`, and the reshaped identifier column.

5.5 Cross-file Consistency Rules

1. **Unit-name integrity:** columns used as unit identifiers across inflow, availability, and technology files must be identical strings. In this repository, `unit_name` from `genco_data_HR_sdewes.xml` is the reference key.
2. **Time-grid integrity:** all time-dependent files should share the same year set and full period set.
3. **Fuel-label integrity:** fuel labels must align across genco, thermal, fuel-cost, and industry tables.
4. **Heat/cooling market integrity:** CHP heat-market names and DH mapping labels must be consistent.
5. **Technology-key integrity:** technology names in `flex_tech` must also exist in `tech_cost_years` and, where applicable, `COP_HP`.

Mandatory matching map

The table below summarizes the minimum identifier matches that must hold across files before running the model. This is the most important practical checklist for avoiding preprocessing failures, silent data loss in merges, or incorrectly parameterized units.

Reference file/key	Must match with	Matching field(s)	Match type	Why it matters / failure mode
`genco_data_HR_sdewes.xml`: `unit_name`	`ncre_aval_factor_HR_2020_2050_sdewes.xml` column headers	`unit_name`, plus `year`, `period`	exact string match	Required for assigning availability factors to non-dispatchable units. If the name does not match, the renewable or HROR time series will not merge correctly into $\texttt{aval\_factor\_plant}_{u,t,y}$.
`genco_data_HR_sdewes.xml`: `unit_name` of `HDAM`/`HPHS` units	`scaled_inflows_HR_2020_2050_sdewes.xml` column headers	`unit_name`, plus `year`, `period`	exact string match	Required for assigning inflows to hydro reservoir and pumped-hydro units. A mismatch prevents the hydro time series from being attached to the intended units.
`genco_data_HR_sdewes.xml`: `fuel_type`	`fuel_cost_2020_2050_sdewes.xml` column headers	`fuel_type`, plus `year`, `period`	exact label match	Required for variable-cost calculation. The code explicitly stops if a unit cannot find a matching fuel-price series.

Reference file/key	Must match with	Matching field(s)	Match type	Why it matters / failure mode
genco_data_HR_sdewes.xml: unit_name where CHPType=='Y'	heat_demand_HR_2020_2050_sdewes.xml column headers	unit_name, plus year, period	exact string match	CHP units are linked to heat-demand columns by name. If the CHP unit name is absent or inconsistent, the heat-demand assignment becomes invalid.
cooling_demand_HR_2020_2050_sdewes.xml market labels	heat-pump and DH market logic from flex_tech_HR_2020_2050_sdewes.xml	market names, plus year, period	exact string match	Cooling balances rely on consistent market naming. A mismatch disconnects cooling demand from the technologies meant to serve it.
demand_2020_2050_sdewes.xml sector columns	internal electricity-demand sector set	demand-sector names, plus year, period	schema match	The electricity-demand table is melted into the internal sector set. Missing or altered sector columns break or distort the demand dictionary.
demand_H2_2020_2050_sdewes.xml sector columns	internal hydrogen-demand sector set	demand-sector names, plus year, period	schema match	Hydrogen demand is built by melting sector columns. Missing sector names lead to incomplete $\texttt{H2\_demand}_{s,t,y}$ coverage.
heat_demand_HR_2020_2050_sdewes.xml: Industry_HT	flex_tech_HR_2020_2050_sdewes.xml sheet Industry	shared industrial-demand logic by year, period, fuel labels	semantic match	The industrial heat-demand trajectory and the industrial fuel-switching table must refer to the same industrial block. Inconsistency distorts fuel-switching bounds and hydrogen substitution.
flex_tech_HR_2020_2050_sdewes.xml: sheet Industry Fuel_source	fuel_cost_2020_2050_sdewes.xml column headers	fuel label, plus year, period	exact label match	Used to build industry fuel prices and endogenous fuel shares. A mismatch means the corresponding industrial fuel cannot be priced correctly.
flex_tech_HR_2020_2050_sdewes.xml: sheet Industry Fuel_source	same sheet CO2_factor rows and internal industry_fuels set	fuel label	exact label match	Required so each industrial fuel has both price and CO2 intensity for fuel-switching and emissions accounting.
flex_tech_HR_2020_2050_sdewes.xml: sheet flex_tech unit_name	sheet tech_cost_years unit_name	unit_name, year	exact string match	Cost trajectories for flex technologies assume synchronized technology names across sheets. A mismatch breaks CAPEX/OPEX assignment by technology-year pair.
flex_tech_HR_2020_2050_sdewes.xml: sheet flex_tech heat-pump names	sheet COP_HP columns	heat-pump identifier, plus year, period	exact string match	Required to generate $\texttt{HP\_COP\_period}_{h,t,y}$. If the COP table does not match the heat-pump names, the conversion efficiency path is incomplete.
import_export_HR_2020_2050_sdewes.xml	all time-series files	period; where applicable year	time-grid match	Import bounds are applied by period. Missing or inconsistent periods create inconsistent balance equations across datasets.

Reference file/key	Must match with	Matching field(s)	Match type	Why it matters / failure mode
All time-series files	model horizon	`year`, `period` coverage	completeness match	Every active model year-period pair must exist in demand, H2 demand, heat, cooling, EV, inflow, and availability inputs. Missing rows cause partial dictionaries and undefined behavior in constraint construction.

In the local preprocessing code, both `scaled_inflows_HR_2020_2050_sdewes.xml` and `ncre_aval_factor_HR_2020_2050_sdewes.xml` are first reshaped into a long format with the key columns `year`, `period`, and `unit_name`, and then merged with the genco table. As a consequence, hydro inflow columns must exactly match the `unit_name` of `HDAM` and `HPHS` units, and renewable availability columns must exactly match the `unit_name` of the non-dispatchable generation units that use those profiles. Any mismatch in spelling, capitalization, whitespace, or naming convention breaks that join and prevents the corresponding time series from being assigned correctly.

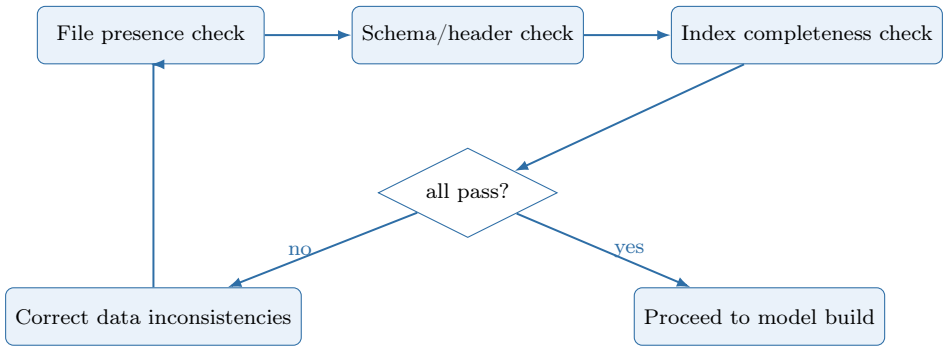

Figure 8: Recommended pre-run validation logic for robust scenario execution.

5.6 Pre-run Data Validation Checklist

- Are all required files present and correctly referenced in `main.py`?
- Are all years present in every time-dependent file?
- Are all periods complete within each year?
- Are there duplicated records for the same identifier tuple?
- Are there missing values in key fields (`unit_name`, `year`, `period`, capacity, efficiency)?
- Are scenario switches compatible with available data (reserves, hydrogen, heat pumps)?

6 How H2RES Represents the Energy System

This section is written for first-time users who need to understand the logic of the model before working through traceability tables and equation appendices. The objective is not to restate the full mathematical formulation, but to show how data, physical flows, and decision variables connect across the main subsystems.

6.1 Integrated system view

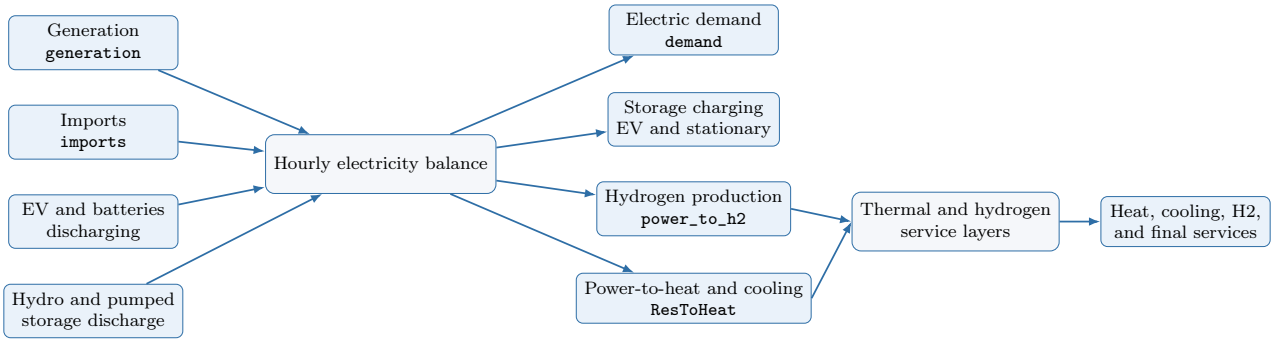

Figure 9: Integrated system view of the main carrier interactions represented in H2RES.

Purpose: give a first-pass view of how electricity sits at the center of the model and feeds storage, hydrogen, and heat/cooling service layers.
Read it as: the left-hand side contains hourly electric injections, the center enforces the electricity balance, and the right-hand side shows where electricity is consumed directly or transformed into other energy carriers.
Key variables: $\texttt{generation}_{u,t,y}$, $\texttt{imports}_{t,y}$, $\texttt{power\_to\_h2}_{t,y}$, $\texttt{ResToHeat}_{t,y}$, $\texttt{ev\_sto\_in/out}_{t,y}$, $\texttt{stat\_sto\_in/out}_{b,t,y}$.
Linked model blocks: generation and investment, electricity balance, EV storage, stationary batteries, hydrogen and fuel cells, heat/cooling and CHP.

6.2 How input data becomes model structure

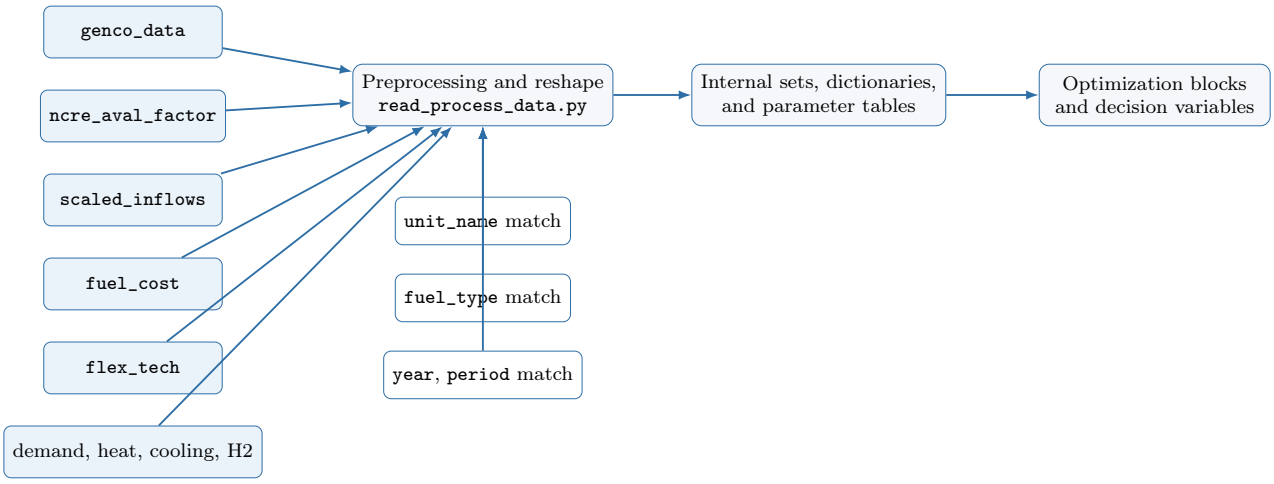

Figure 10: Data-engineering logic from source files to model-ready parameters and sets.

Purpose: show that H2RES is built through preprocessing joins, not by reading one monolithic table.
Read it as: the raw files on the left are harmonized in `read_process_data.py`; the joins only succeed when identifiers and time axes are consistent.
Key identifiers: `unit_name`, `fuel_type`, `year`, and `period`. These keys determine whether operational profiles, costs, and technical attributes are assigned to the correct asset or demand block.

Linked manual sections: Input Data Engineering Guide, Cross-file Consistency Rules, and Appendix D.

6.3 Electricity balance logic

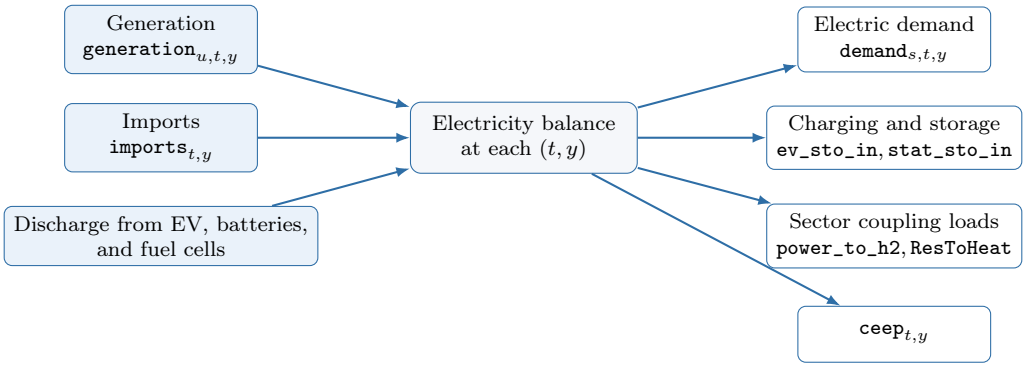

Figure 11: Conceptual reading of the hourly electricity balance in H2RES.

Purpose: make clear that H2RES is organized around one central adequacy equation at every hour and model year.
Read it as: all electricity injections must be matched by direct electric demand, storage charging, conversion to hydrogen or heat, and any excess of electrical production represented by `ceep`.
Key variables: `generation`$_{u,t,y}$, `imports`$_{t,y}$, `ev_sto_out`$_{t,y}$, and `stat_sto_out`$_{b,t,y}$ define the main hourly injections, while `power_to_h2`$_{t,y}$, `ResToHeat`$_{t,y}$, and `ceep`$_{t,y}$ sit on the withdrawal side of the balance.
Why this matters for new users: many apparent anomalies in outputs are not solver issues; they are usually consequences of how charging loads, hydrogen production, or heat electrification appear on the right-hand side of this balance.

6.4 Heat and cooling service logic

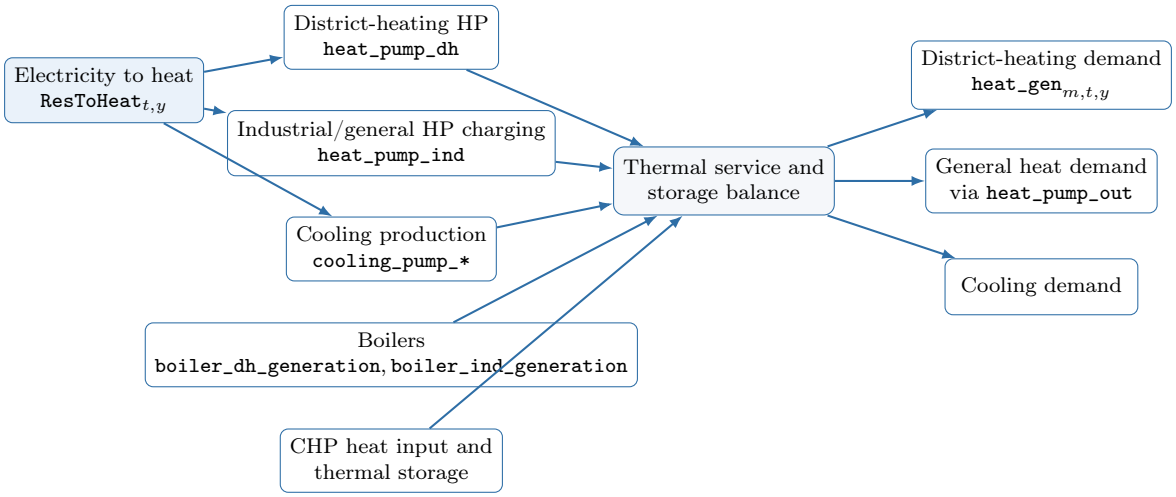

Figure 12: Heat and cooling service logic, including heat pumps, boilers, CHP, and thermal storage.

Purpose: clarify the most confusing part of the model for new users: electricity does not serve heat demand directly, but through heat-pump conversion, CHP coupling, boiler support, and thermal storage release.
Read it as: `ResToHeat` is the aggregate electric input to thermal conversion; that input is distributed across district-heating heat pumps, industrial or general heat pumps, and cooling devices, then combined with boilers and CHP-linked heat to satisfy thermal services.

Frequently confused variables:

Variable	Unit	Practical meaning for a new user
$\texttt{heat\_pump\_ind}_{h,t,y}$	MW_{th}	Heat added into the industrial/general thermal-storage pathway by a heat pump before final service is released.
$\texttt{heat\_pump\_out}_{h,t,y}$	MW_{th}	Heat withdrawn from that heat-pump thermal pathway to satisfy general heat demand.
$\texttt{heat\_pump\_sto\_soc}_{h,t,y}$	MWh_{th}	State of charge of the heat-pump-related thermal buffer over time.
$\texttt{heat\_gen}_{m,t,y}$	MW_{th}	District-heating service delivered by CHP-linked thermal systems to meet market-specific heat demand.
$\texttt{boiler\_ind\_generation}_{b,t,y}$	MW_{th}	Boiler heat sent directly to the non-district industrial/general heat service branch.

Linked model blocks: heat/cooling and CHP, industrial fuel switching, and the thermal parts of Appendix C.

6.5 Hydrogen subsystem logic

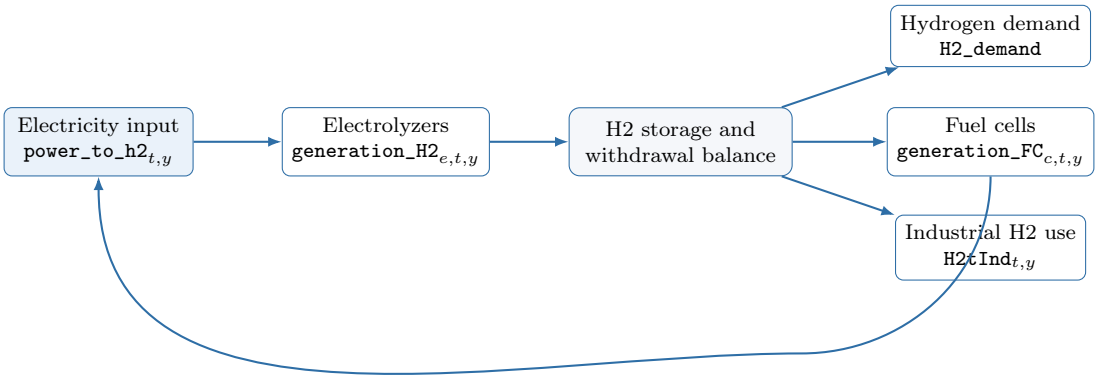

Figure 13: Hydrogen subsystem from electric conversion to storage, end use, and fuel-cell return.

Purpose: show that hydrogen in H2RES is not an isolated demand block; it is an intertemporal energy carrier connected back to electricity through fuel cells.
Read it as: electricity first feeds electrolysis, hydrogen can then be stored, and stored hydrogen is allocated among exogenous hydrogen demand, industrial substitution, and electricity re-generation through fuel cells.
Key variables: $\texttt{power\_to\_h2}_{t,y}$ is the electric load used for hydrogen conversion; $\texttt{generation\_H2}_{e,t,y}$, $\texttt{h2\_sto\_soc}_{s,t,y}$, and $\texttt{h2\_sto\_out}_{s,t,y}$ describe hydrogen production and inventory; $\texttt{generation\_FC}_{c,t,y}$ and $\texttt{H2tInd}_{t,y}$ capture the main downstream uses.
Why this matters for new users: high electricity demand from electrolysis can be entirely consistent with low direct electric demand growth, because the hydrogen branch is represented as an additional conversion load rather than a conventional demand sector.

6.6 Storage logic across carriers

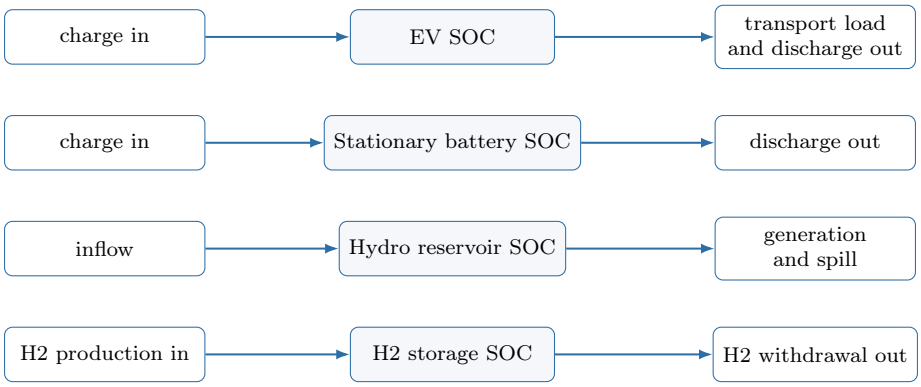

Figure 14: Common state-of-charge logic across EV, stationary battery, hydro, and hydrogen storage.

Purpose: help new users recognize that several subsystems differ in physics but share the same intertemporal modeling pattern: previous inventory plus inflow minus outflow, subject to capacity and terminal conditions.
Read it as: each storage family has its own carrier and constraints, but all of them are modeled as state variables coupled across hours and years.
Key state variables: $\texttt{ev\_sto\_soc}_{t,y}$, $\texttt{stat\_sto\_soc}_{b,t,y}$, $\texttt{storage\_level}_{u,t,y}$, $\texttt{h2\_sto\_soc}_{s,t,y}$.
Practical implication: when a scenario changes storage size, flexibility, or inflow assumptions, the effect is usually spread over multiple hours rather than appearing as a single-period response.

7 Adapting H2RES to a New Case Study

This section translates common user tasks into the files and assumptions that must be changed. It is intentionally task-oriented rather than file-oriented, because new users usually start from a research question, not from the internal repository structure.

7.1 What usually needs to be changed

Task	Main file(s) to edit	Priority	Why it matters	Typical failure if wrong
Change the scenario name, switches, or policy path	`main.py`	Mandatory	Defines the active scenario, optional blocks, and core policy assumptions for the run.	Wrong module activation, overwritten result folders, or unintended policy behavior.
Change electricity demand trajectories	`demand_2020_2050_sdewes.xml`	Mandatory for a new load case	Alters the central electricity balance and adequacy requirements.	Missing periods, distorted load levels, or infeasibility.
Change generation asset portfolio	`genco_data_HR_sdewes.xml`	Mandatory when units differ	Defines the set of units, their technical attributes, and their classification into model subsets.	Missing units in availability/inflow files, wrong technology subsets, or inconsistent costs.
Change renewable availability profiles	`ncre_aval_factor_HR_2020_2050_sdewes.xml`	Mandatory for new RES units or profiles	Controls non-dispatchable hourly production envelopes.	Unit-name mismatch or silent misassignment of hourly profiles.

Task	Main file(s) to edit	Priority	Why it matters	Typical failure if wrong
Change hydro inflows	`scaled_inflows_HR_2020_2050_sdewes.xml`	Mandatory for new hydro cases	Drives reservoir and pumped-hydro intertemporal availability.	Hydro units without inflow series or incorrect water-energy behavior.
Change import/export assumptions	`import_export_HR_2020_2050_sdewes.xml`	Case-dependent	Alters interconnection limits and export-aware balance behavior.	Unrealistic adequacy support or broken export logic.
Change fuel prices	`fuel_cost_2020_2050_sdewes.xml`	Usually mandatory	Affects dispatch costs, industrial-fuel economics, and thermal operating cost.	Fuel-label mismatch or distorted merit-order outcomes.
Change heat, cooling, or hydrogen demand	`heat_demand_HR_2020_2050_sdewes.xml`, `cooling_demand_HR_2020_2050_sdewes.xml`, `demand_H2_2020_2050_sdewes.xml`	Mandatory for sector-coupling studies	Changes service obligations in thermal and hydrogen blocks.	Missing market labels, broken closure constraints, or implausible coupling results.
Change EV, H2, storage, heat-pump, or boiler technology assumptions	`flex_tech_HR_2020_2050_sdewes.xml` and `main.py`	Usually mandatory for flexibility studies	Controls technology costs, efficiencies, limits, DH mapping, and industrial fuel shares.	Technology-key mismatch across sheets or inconsistent bounds in coupled sectors.
Change technology learning or surviving-capacity trajectories	`main.py`	Optional but often important	Governs intertemporal evolution of investment attractiveness and inherited stock.	Scenario narrative inconsistent with long-term capacity behavior.

7.2 Recommended order when adapting the model

1. Change only high-level scenario settings in `main.py` and rerun the baseline data package.
2. Replace or update core demand and supply master tables.
3. Update profile files that must match those master tables by identifier and time grid.
4. Update sector-coupling workbooks (`flex_tech`, heat, cooling, hydrogen) only after core identifiers are stable.
5. Re-run the validation logic in Sections 5, 8, 10, and Appendix D before interpreting any new scenario result.

7.3 What should usually *not* be changed first

- Do not start by modifying `Build_model.py`; most new-case adaptations belong to inputs or `main.py`, not to equations.
- Do not rename units, fuels, or markets in only one file; identifier changes must propagate through every matching file.
- Do not tighten several policy trajectories simultaneously before a relaxed case runs correctly.

8 Traceability: Input Data to Model Blocks

8.1 Block-Level Traceability Matrix

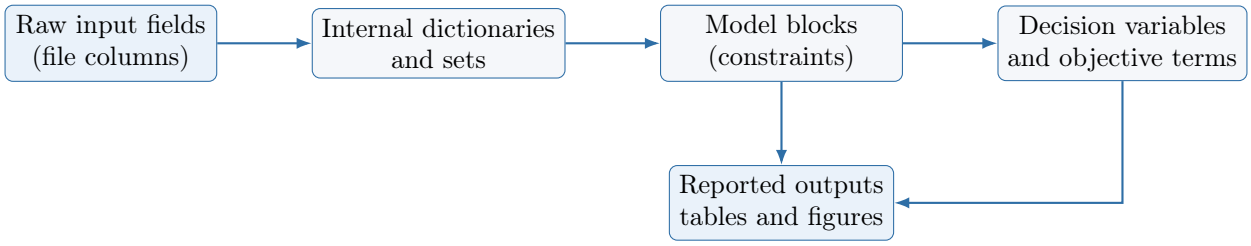

Figure 15: Traceability chain from source data to optimization outputs.

Model block	Primary data sources	Representative internal parameters	Main decision variables	Main constraints in code
Generation and investment	genco, fuel cost, availability	`max_load`, `cap_factor`, `var_cost`, `decom_rate`	`generation`, `cap_inv`	`max_output*`, `max_inv_unit`
Electricity balance and imports	demand, import-export, EV/storage coupling	`demand`, `import_ntc`	`imports`, `ceep`	`meet_demand`, `import_lim`
Hydrogen and fuel cells	flex_tech, H2 demand, tech-cost-years	`H2_eff`, `H2_demand`, `FC_eff`	`generation_H2`, `h2_sto_soc`, `generation_FC`	`Power_to_h2`, `h2_storage*`, `max_output_FCell`
EV and stationary storage	EV profile, flex_tech, tech-cost-years	`ev_availability`, `Sta_sto_eff`, `Sta_sto_max_cap`	`ev_sto_*`, `stat_sto_*`	`ev_storage*`, `stat_storage*`
Heat/cooling and CHP	heat demand, cooling demand, flex_tech, CHP fields in genco	`HP_COP_period`, `CHPPowerToHeat`, `STOCapacity_heat`	`heat_pump_*`, `boiler_*`, `heat_gen`, `heat_sto_level`	`heat_pump*`, `meet_heat_demand*`, `meet_cooling_demand*`
Hydro storage dynamics	inflow file + genco hydro fields	`hdam_inflow`, `hphs_inflow`, storage capacities	`storage_level`, `sto_out_flow`	`storage_level*`, `hphs_storage*`
Policies and emissions	main policy vectors, genco/thermal/industry CO2 data	`rps`, `CO2_limit`, `CO2_factor`, `Fuel_CO2`	`CO2_total*`, all operational vars	`rps`, `CO2_emissions*`
Primary/secondary reserves (optional)	reserve flags in genco/flex_tech	reserve unit sets, `reserve_factor`	reserve up/down variables by technology	`primary_*`, `secondary_*` constraint families

8.2 Critical Parameter Traceability Examples

Internal parameter	Source file	Source field(s)	Dimensions	Why it matters
`var_cost`	fuel cost + genco	fuel price, `cost_no_fuel`, efficiency	(u, t, y)	Directly drives dispatch merit order and objective cost.
`HP_COP_period`	flex_tech (`COP_HP`)	HP columns by `year,period`	(h, t, y)	Converts electrical input into heat/cooling output.
`fuel_share_max`	flex_tech (`Industry`) + fuel prices + CO2 price	`Industry_HT`, fuel prices, `CO2_factor`	(t, y, f)	Governs endogenous industrial fuel switching.
`import_ntc`	import-export file	`Import_capacity`	(t)	Caps import dependence and affects adequacy margins.
`TechChange`	`main.py` vectors + unit filters	`TechChangeSolar`, `TechChangeWind`	(u, y) for wind/solar	Alters intertemporal attractiveness of RES investments.

8.3 Comprehensive Parameter Hand-off Table

This table expands the parameter traceability requested for implementation-level work. It focuses on parameters explicitly created/processed in `read_process_data.py` and consumed in `Build_model.py`.

Internal parameter (model notation)	Source file/sheet	Source fields	Indices in model	Unit	Main consuming blocks
`demand`$_{s,t,y}$	`demand_2020_2050_sdewes.xml`	sector columns, `year`, `period`	(s,t,y)	MW$_e$	Electricity balance, total load, adequacy and CEEP bounds
`H2_demand`$_{s,t,y}$	`demand_H2_2020_2050_sdewes.xml`	sector columns, `year`, `period`	(s,t,y)	MW$_{H2}$	Hydrogen withdrawal balance and reserve feasibility margins
`heat_demand`$_{m,t,y}$	`heat_demand_HR_2020_2050_sdewes.xml`	market columns, `general_demand`, `year`, `period`	(m,t,y)	MW$_{th}$	CHP heat balance, district heating closure, general heat adequacy
`cooling_demand`$_{m,t,y}$	`cooling_demand_HR_2020_2050_sdewes.xml`	market columns, `general_cooling`, `year`, `period`	(m,t,y)	MW$_{cool}$	Cooling adequacy constraints in heat-pump/cooling block
`var_cost`$_{u,t,y}$	genco + `fuel_cost_2020_2050_sdewes.xml`	`cost_no_fuel`, fuel prices, `efficiency`	(u,t,y)	EUR/MWh$_e$	Objective operating cost for electricity generation
`heat_var_cost`$_{b,t,y}$	thermal_tech + fuel cost	fuel price, thermal efficiency	(b,t,y)	EUR/MWh$_{th}$	Objective operating cost for thermal boilers
`industry_fuels_price`$_{f,t,y}$	`fuel_cost_2020_2050_sdewes.xml`	fuel columns by time	(f,t,y)	EUR/MWh$_{fuel}$	Industrial fuel expenditure term in objective
`fuel_share_max`$_{t,y,f}$	`flex_tech...` sheet `Industry` + fuel cost + CO2 price	`Industry_HT`, `CO2_factor`, fuel price	(t,y,f)	p.u.	Industrial fuel-switch constraint upper bounds
`import_ntc`$_t$	`import_export_HR_2020_2050_sdewes.xml`	`Import_capacity`	(t)	MW$_e$	Import upper bound and adequacy flexibility
`exports`$_t$	`import_export_HR_2020_2050_sdewes.xml`	`total_exports`	(t)	MW$_e$	Export-aware power-balance variant
`max_load`$_u$	`genco_data_HR_sdewes.xml`	`cap_mw`	(u)	MW$_e$	Capacity envelopes, reserves, ramping, hydro energy conversion
`cap_factor`$_u$	`genco_data_HR_sdewes.xml`	`cap_factor`	(u)	p.u.	Dispatch upper bounds for thermal/hydro and CHP extraction logic
`cap_inv_cost`$_u$	`genco_data_HR_sdewes.xml`	`cap_inv_cost`	(u)	EUR/MW	Capacity expansion CAPEX in objective
`ramp_up_rate`$_u$, `ramp_down_rate`$_u$	`genco_data_HR_sdewes.xml`	`RampUpRate`, `RampDownRate`	(u)	p.u./h	Inter-hour generation ramp constraints
`CO2_factor`$_u$	`genco_data_HR_sdewes.xml`	`CO2Intensity`	(u)	tCO2/MWh$_e$	Power-sector annual emissions accounting and carbon cost
`H2_eff`$_e$	`flex_tech...` sheet `flex_tech`	electrolyzer efficiency	(e)	p.u.	Power-to-hydrogen conversion, reserve feasibility for electrolyzers
`FC_eff`$_c$	`flex_tech...` sheet `flex_tech`	fuel-cell efficiency	(c)	p.u.	Fuel-cell generation and hydrogen-use coupling
`HP_COP_period`$_{h,t,y}$	`flex_tech...` sheet `COP_HP`	COP columns by `year,period`	(h,t,y)	p.u.	Electric-to-heat conversion and reserve flexibility for HP
`Sta_sto_eff`$_b$	`flex_tech...` sheet `flex_tech`	storage efficiency	(b)	p.u.	Stationary battery SOC recursion and electric balance terms
`h2_sto_init`$_s$, `H2_sto_max_cap`$_s$	`flex_tech...` sheet `flex_tech`	`cap_mw`, `max_cap` for H2 storage	(s)	MWh$_{H2}$	Hydrogen SOC bounds and storage expansion limit
`hdam_inflow`$_{u,t,y}$, `hphs_inflow`$_{u,t,y}$	`scaled_inflows_HR_2020_2050_sdewes.xml`	unit inflow columns + time keys	(u,t,y)	p.u. inflow	Reservoir/pumped-hydro dynamic balance and availability
`aval_factor_plant`$_{u,t,y}$	`ncre_aval_factor_HR_2020_2050_sdewes.xml`	plant availability columns + time keys	(u,t,y)	p.u.	Equality dispatch for non-dispatchable units (wind/solar/HROR)

Internal parameter (model notation)	Source file/sheet	Source fields	Indices in model	Unit	Main consuming blocks
$\texttt{TechChange}_{u,y}$	`main.py` + unit filters from genco	`TechChangeSolar`, `TechChangeWind`	(u, y)	p.u. factor	Technology-specific investment attractiveness in objective
$\texttt{NPV}_y$	`main.py`	discount vector	(y)	p.u.	All intertemporal objective components
$\texttt{rps}_y$	`main.py`	annual policy trajectory	(y)	p.u.	Renewable portfolio share policy constraint
$\texttt{CO2\_limit}_y$, $\texttt{CO2\_price}_y$	`main.py`	annual policy trajectories	(y)	tCO2/year, EUR/tCO2	Emissions cap constraints and carbon-cost terms
$\texttt{reserve\_factor}_u$	genco reserve flags (`PrimaryReserve`)	`Stab_factor`	(u)	p.u.	Reserve-down technical feasibility for dispatchable units
$\texttt{ev\_availability}_{t,y}$, $\texttt{ev\_transp\_load}_{t,y}$, $\texttt{ev\_Grid\_P\_max}_y$, $\texttt{ev\_sto\_max}_y$	`ev_transp_load.xml` + `main.py`	profile columns + EV assumptions	(t, y), (y)	p.u., MW_e, MWh_e	EV SOC recursion, charging/discharging limits, electricity balance

8.4 Decision Variables Organized by Model Block

The table below complements Section 3 by grouping variables according to the operational blocks used in `Build_model.py`.

Model block	Main decision variables (original notation)	Main indices	Functional role
Generation and investment	$\texttt{generation}_{u,t,y}$, $\texttt{cap\_inv}_{u,y}$, $\texttt{imports}_{t,y}$, $\texttt{ceep}_{t,y}$	(u,t,y), (u,y), (t,y)	Core electric dispatch and capacity expansion with adequacy slack
Hydrogen and fuel cells	$\texttt{generation\_H2}_{e,t,y}$, $\texttt{h2\_sto\_soc}_{s,t,y}$, $\texttt{h2\_sto\_out}_{s,t,y}$, $\texttt{generation\_FC}_{c,t,y}$, $\texttt{cap\_inv\_FC}_{c,y}$, $\texttt{h2\_electrolysis\_max\_cap}_{e,y}$	(e,t,y), (s,t,y), (c,t,y), (c,y), (e,y)	Couples electricity, hydrogen conversion, storage, and re-electrification
EV storage	$\texttt{ev\_sto\_in}_{t,y}$, $\texttt{ev\_sto\_out}_{t,y}$, $\texttt{ev\_sto\_soc}_{t,y}$	(t,y)	Transport electrification flexibility through managed charging/discharging
Stationary batteries	$\texttt{stat\_sto\_in}_{b,t,y}$, $\texttt{stat\_sto\_out}_{b,t,y}$, $\texttt{stat\_sto\_soc}_{b,t,y}$, $\texttt{stat\_sto\_cap\_inv}_{b,y}$	(b,t,y), (b,y)	Short-term balancing and intertemporal storage capacity expansion
Heat/cooling and CHP	$\texttt{heat\_pump\_dh}_{h,m,t,y}$, $\texttt{heat\_pump\_ind}_{h,t,y}$, $\texttt{cooling\_pump\_dh}_{h,m,t,y}$, $\texttt{cooling\_pump\_ind}_{h,t,y}$, $\texttt{heat\_pump\_out}_{h,t,y}$, $\texttt{heat\_pump\_sto\_soc}_{h,t,y}$, $\texttt{boiler\_dh\_generation}_{b,m,t,y}$, $\texttt{boiler\_ind\_generation}_{b,t,y}$, $\texttt{heat\_gen}_{m,t,y}$, $\texttt{heat\_sto\_level}_{m,t,y}$	(h,m,t,y), (h,t,y), (b,m,t,y), (b,t,y), (m,t,y)	Closes thermal and cooling balances and links them to electricity via COP/CHP coupling
Industrial fuel switching	$\texttt{Fuel\_ind}_{f,t,y}$, $\texttt{H2tInd}_{t,y}$	(f,t,y), (t,y)	Endogenous industrial fuel-mix allocation and hydrogen substitution
Hydro reservoirs and pumped hydro	$\texttt{storage\_level}_{u,t,y}$, $\texttt{sto\_out\_flow}_{u,t,y}$, $\texttt{generation}_{u,t,y}$	(u,t,y)	Intertemporal water-energy accounting with spill and terminal constraints
Reserves (optional)	$\texttt{primary\_*}_{\cdot,t,y}$, $\texttt{secondary\_*}_{\cdot,t,y}$, technology-specific reserve variables for units/storage/FC/ELY/HP	$(\cdot,t,y)$	Up/down balancing capability under primary and secondary reserve modules
Emissions accounting and policy	$\texttt{CO2\_total}_{y}$, $\texttt{CO2\_total\_heat}_{y}$, $\texttt{CO2\_total\_industry}_{y}$	(y)	Tracks sectoral annual emissions and enforces cap trajectories

8.5 Unit Convention and Dimensional Consistency

H2RES is solved on an hourly time horizon. For this reason, many balance equations use hourly-average power variables that are numerically equivalent to energy over one hour. To avoid ambiguity in publication-grade analysis, use the convention below consistently.

Quantity family	Unit convention	Practical interpretation	Validation rule
Electric generation and flows (`generation`, `imports`, `ceep`, storage power)	MW_e (hourly average)	Average electric power in one model period (1 hour).	In each hourly electricity balance, all additive terms must use MW_e-equivalent magnitudes.
Electric storage state variables (`ev_sto_soc`, `stat_sto_soc`, hydro `storage_level`)	MWh_e	Stored electric energy at period boundary.	SOC recursion must combine previous SOC and hourly inflow/outflow power terms consistently over 1-hour step.
Heat and cooling flows	MW_{th}, MW_{cool}	Hourly thermal/cooling service rate.	Heat/cooling demand closure constraints must be unit-homogeneous at each (t, y).
Hydrogen production and storage	MW_{H2}, MWh_{H2}	Hydrogen power-rate equivalent and inventory.	Conversion constraints must apply `H2_eff`/`FC_eff` to map electric and hydrogen domains.
Costs and prices	EUR, EUR/MWh, EUR/MW, EUR/tCO2	Objective components under annual discounting (`NPV`).	Every objective term must be reduced to EUR before aggregation.
Emissions	tCO2, tCO2/MWh	Sectoral annual emissions and intensities.	Annual cap uses consistent sum: power + heat + industry emissions in tCO2/year.
Dimensionless controls (`rps`, reserve factors, availability, COP, efficiencies)	p.u.	Fractions or coefficients.	Must multiply physically compatible quantities; never add directly to power/energy terms.

8.6 Embedded Model Traceability Within This Manual

Model traceability in this manual is organized around three complementary layers:

- The main body explains the user workflow, data flow, system logic, traceability, and validation sequence.
- Appendix C maps selected implemented constraint groups to their mathematical meaning and interpretation.
- Appendix D documents the data-matching logic required for preprocessing consistency.

Together, these components provide a concise link between model formulation, implementation structure, and input-data consistency.

9 Interpreting Outputs

9.1 Where results are written

Outputs are written to:

`results/<scenario_name>/`

where `<scenario_name>` is the value of `scen` in `main.py`.

9.2 Main output families

- Generation by fuel and time (`gen_by_fuel_wide_GWh.xlsx`).
- Capacity expansion plots and tables (`cap_inv.png`, investment tables).
- Hydro storage and spillage trajectories.
- Heat and cooling production by technology and market.

- EV and stationary storage operation.
- Hydrogen production/storage and fuel-cell utilization.
- Consolidated text report (`report.txt`) with key summary metrics.

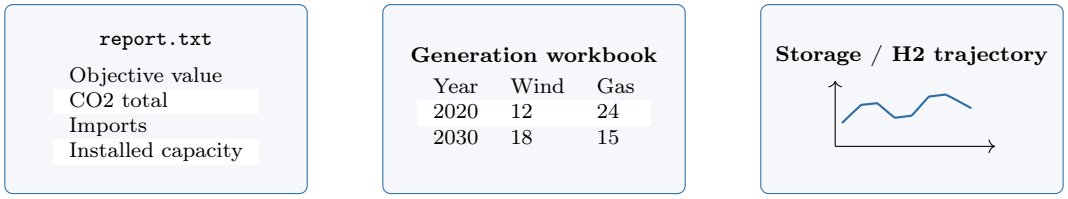

Figure 16: Typical visual forms of key outputs: text summary, workbook-style aggregate table, and time-series trajectory. Exact values change by scenario, but the artifact types remain similar.

9.3 What to inspect first after a run

Output file or family	What it contains	First question it answers	When to inspect it
`report.txt`	Consolidated summary metrics and high-level run information	Did the run complete and what are the headline results?	Always inspect first.
Generation summary workbook	Technology/fuel generation summary across the horizon (`gen_by_fuel_wide_GWh.xls`	How did the electric supply mix behave?	Inspect immediately after `report.txt`.
Hydro-storage workbooks	Reservoir and spillage trajectories (`StorageLevel_Hydro.xlsx`, `spillage_hydro_storage.x`	Is hydro behaving intertemporally as expected?	Inspect whenever hydro or adequacy is important.
`ev_df.xlsx` and stationary-storage tables	EV and battery operation over time	Is short-term flexibility being used plausibly?	Inspect in storage or flexibility studies.
Heat and boiler workbooks	Thermal-service production and conversion pathways (`Heat_hp_*.xlsx`, `boiler_*.xlsx`)	Which technologies are serving heat and cooling demand?	Inspect in sector-coupling or DH studies.
Hydrogen workbooks	Hydrogen production, storage, and fuel-cell use (`h2_*.xlsx`, `FC_*.xlsx`, `P2X.xlsx`)	Is hydrogen behaving mainly as load, storage, or re-electrification?	Inspect whenever hydrogen blocks are active.
`cap_inv.png` and investment tables	Capacity-expansion outcomes	What new capacity did the model choose and when?	Inspect in planning or policy scenarios.

9.4 Recommended first-pass reading order

For a new user, the most efficient order is:

1. `report.txt` for overall sanity.
2. Generation summary to understand the electric supply mix.
3. Storage and hydro files to check intertemporal behavior.
4. Heat/cooling and hydrogen outputs only after the electric backbone looks credible.
5. Capacity-expansion outputs last, because they are easier to interpret once dispatch and service balances already make sense.

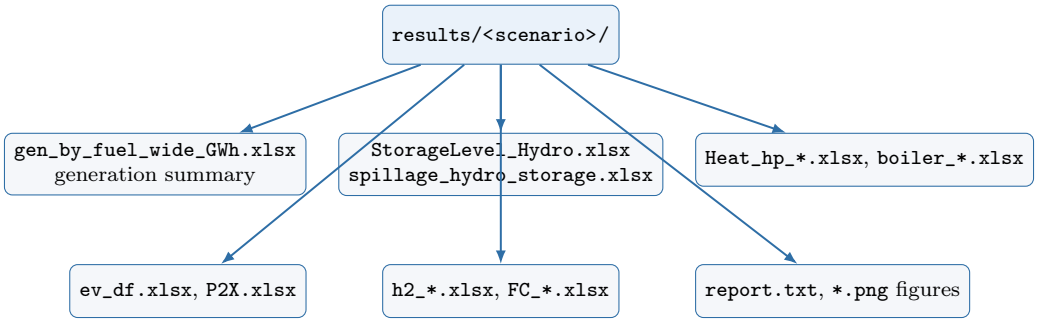

Figure 17: Output artifact map for post-processing and result interpretation.

9.5 Interpretation

- Zero output can be expected when a module is disabled by scenario switches.
- Missing files can occur when `save_csv=False`.
- Apparent anomalies often come from input inconsistencies before they come from solver behavior.

10 Troubleshooting Matrix

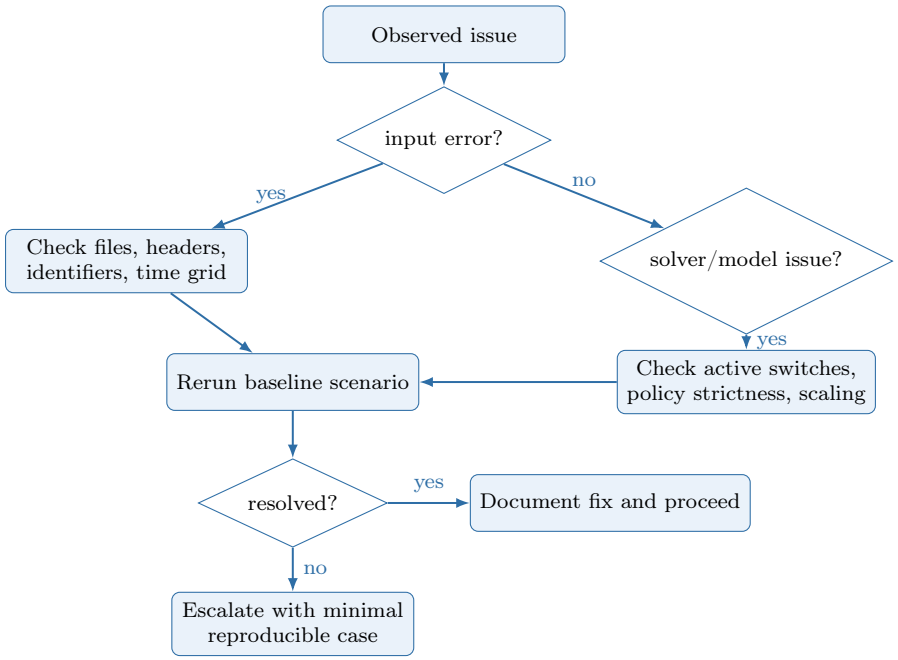

Figure 18: Troubleshooting decision path for rapid diagnosis and recovery.

Issue pattern	Likely cause	Suggested check	Corrective action
Model infeasible	Inconsistent demand/capacity/policy assumptions	Inspect solver status prints and tightest policy settings	Start from relaxed policy test case, then retighten incrementally
KeyError during model build	Missing index tuples in dictionaries	Check completeness of (t, y) and (u, t, y) combinations after preprocessing	Fill missing rows; enforce complete hourly grids
Fuel price stop in preprocessing	Fuel label mismatch between files	Compare `fuel_type` labels with fuel-cost columns	Harmonize labels or add missing fuel columns
Heat/cooling imbalance behavior	Market-name mismatch or missing general demand columns	Verify `general_demand`, `general_cooling`, CHP market names	Align labels across heat/cooling and DH mapping files
Reserve module errors	Reserve switches active with incomplete reserve-eligible sets	Check Y/N flags and resulting reserve sets from preprocessing	Populate reserve flags or disable reserve switches for baseline run
Outputs overwritten	Same scenario name reused	Check `export_plot.py` folder reset behavior	Archive old run or use a new `scen` label
Numeric issues status	Poor data scaling or extreme parameter values	Inspect magnitude ranges for costs, capacities, and flows	Recheck units and scale assumptions consistently

11 Verification and Validation Protocol

11.1 Structural Validation Before Solve

Apply these checks prior to model construction:

1. Identifier integrity across files (`unit_name`, fuel labels, market labels, technology keys).
2. Full time-grid coverage for all (t, y) required by active modules.
3. No duplicate tuples for primary keys in melted long-format tables.
4. No nulls in mandatory fields used to build dictionaries.
5. Scenario-switch compatibility with data availability (reserves, hydrogen, heat-pump blocks).

11.2 Numerical and Behavioral Validation After Solve

Validation target	Computation approach	Acceptance criterion	Action if failed
Hourly electricity balance residual	Recompute LHS-RHS for each (t, y).	Absolute residual $\leq 10^{-6}$ in model units.	Inspect indexing completeness, sign convention, and optional module activation logic.
Hydrogen subsystem residual	Check production-storage-demand consistency for each (t, y).	Absolute residual $\leq 10^{-6}$.	Recheck `H2_eff`, demand indexing, and storage capacity dictionaries.
Heat/cooling demand closure	Recompute heat and cooling equalities by market and period.	Residual near solver tolerance.	Verify DH mapping, `general_demand`, and COP table completeness.
Policy compliance flags	Evaluate RPS and CO2 constraints against reported totals.	Constraints respected when active; unconstrained when disabled.	Verify boolean switches and policy vectors in `main.py`.
Intertemporal SOC sanity	Inspect SOC trajectories and terminal conditions.	No negative inventory if constrained; terminal rules satisfied.	Recheck SOC bounds and decomposition factors.

11.3 Scenario Toggle Regression Tests

For robust research workflows, keep a minimal regression matrix:

- Baseline: reserves off, moderate policy.
- +Heat/H2 active.
- +Primary reserve active.
- +Secondary reserve active.
- Tight policy stress test (higher RPS and lower CO2 cap).

Track solver status and key outputs across the matrix to detect unintended behavioral drift after data/code changes.

12 PhD-Level Research Practice and Reproducibility

12.1 Minimum reproducibility package per scenario

For each scenario run, store:

- Scenario: all switch values and policy vectors from `main.py`.
- Input dataset snapshot (immutable copy or versioned hash).

- Solver version and parameter settings.
- Output folder and post-processing scripts used.
- Brief run log describing unusual behavior or manual data corrections.

12.2 Reproducible Baseline Mini-Case (Croatia Template)

Use the default repository setup as a reproducibility anchor:

1. Set baseline scenario switches in `main.py` and record them in the run log.
2. Run `python main.py` from repository root.
3. Confirm solver status prints `Status==2` or, only with explicit justification from the solver log, `Status==13`.
4. Verify output folder creation under `results/<scenario>/`.
5. Confirm presence of core artifacts: `report.txt`, generation summary, storage files, heat/H2 outputs.
6. Recompute one-hour checks for electricity and hydrogen balances from exported tables.

Expected baseline behavior:

- No hard infeasibility for default data package.
- Policy constraints bind only when their switches are enabled.
- Module-specific outputs are null/absent when corresponding modules are disabled.

12.3 Recommended experimental protocol

1. Validate baseline data with all optional modules off (reserves off, moderate policy).
2. Activate one complexity layer at a time (heat/hydrogen/reserves).
3. Perform sensitivity analysis on a small set of policy and cost drivers.
4. Report both system-level outputs and binding constraints to explain causality.

12.4 When to modify equations

Equation modifications should start only after:

- data consistency checks pass,
- baseline scenario reproduces expected behavior,
- traceability links (input $\rightarrow$ parameter $\rightarrow$ constraint) are documented.

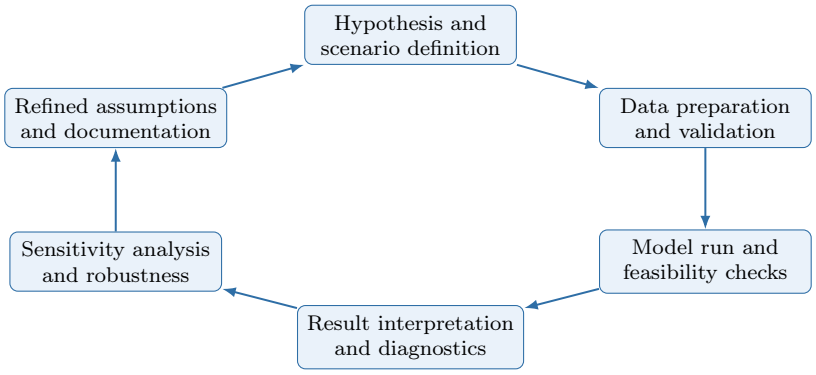

Figure 19: Recommended research cycle using H2RES for transparent iteration.

13 Modeling Assumptions, Implications, and Risks by Block

Model block	Core assumption	Implication for results	Main risk if violated
Electricity	All active loads and couplings are represented in hourly balance.	Dispatch and imports are interpreted as physically feasible hourly system operation.	Hidden or mis-indexed load terms can create false adequacy or false deficits.
Hydrogen subsystem	Efficiency-based conversion and storage recursions are complete.	H2 expansion and FC dispatch are economically comparable with direct electrification pathways.	Mis-specified H2 efficiencies can bias both cost and feasibility.
Heat/cooling subsystem	COP and CHP coupling correctly map electric-thermal interactions.	Heat and cooling service are co-optimized with electricity and fuel switching.	Inconsistent market labels or COP profiles can distort heat-electrification conclusions.
Reservoir and pumped hydro	Inflows and storage bounds represent physical water availability.	Hydro dispatch captures intertemporal opportunity cost of water.	Inflow misalignment by unit/time can produce unrealistic hydro flexibility.
Reserves	Eligible sets and reserve factors represent technology capability.	Reserve costs and capability margins shape flexibility investment and operation.	Incorrect reserve eligibility can overstate balancing security.
Policy blocks	RPS/CO2/CEEP constraints reflect intended trajectories and switches.	Policy sensitivity studies are interpretable and reproducible.	Mis-specified policy vectors lead to misleading transition pathways.

Model block	Core assumption	Implication for results	Main risk if violated
Costing and discounting	Objective terms are monetized consistently in EUR using `NPV`.	Intertemporal trade-offs are coherent across CAPEX/OPEX/policy costs.	Unit mismatch in costs can invert investment rankings.

14 Known Boundaries and Open Items

- Some legacy variable names in code are not self-explanatory and can complicate first-time interpretation.
- Environment bootstrapping commands (dependency installation specification) are not formalized in a repository lockfile.
- Traceability is intentionally summarized at block and selected-constraint level; a future code-auditing appendix could expand this to a full line-by-line implementation map if required.

A Appendix A: Flex-Technology Workbook Sheet Roles

Worksheet	Core fields	Role in model
`flex_tech`	`unit_name`, `Tech_type`, efficiency, capacities, reserve flags	Defines FC, electrolyzers, HP, H2 storage, and stationary storage sets and parameters.
`thermal_tech`	thermal unit fields, efficiency, CO2, decomposition fields	Defines thermal boiler technologies for heat blocks.
`DistrictHeating`	`unit` + DH-market columns	Maps initial DH capacities by technology-market pair.
`tech_cost_years`	`year` + technology columns	Provides annualized CAPEX trajectories for flexible technologies.
`Industry`	`Fuel_source`, `Industry_HT`, `CO2_factor`	Drives industrial fuel-share and fuel-switch logic.
`COP_HP`	`year`, `period`, HP columns	Provides hourly COP profiles used in power-to-heat conversion.

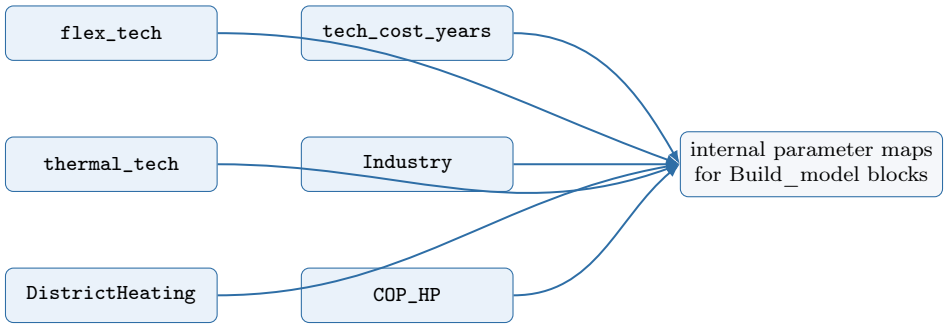

Figure 20: How flex-technology workbook sheets feed model parameter dictionaries.

B Appendix B: Suggested Future Validation Script Logic

An optional `validate_input_data.py` could run:

1. file-existence checks,
2. required column checks per file/sheet,
3. duplicate-key checks (`year,period,...`),
4. missing-value checks in mandatory fields,
5. cross-file name matching (units, fuels, markets, technologies),
6. time-grid completeness checks.

This should remain non-invasive (read-only checks, no data mutation).

C Appendix C: Equation Traceability from `Build_model.py` with User Commentary

This appendix provides an equation-level interpretation of representative implemented constraints.

C.1 Code-to-Equation Mapping (selected core constraints)

Code reference	Constraint/object in code	Simplified equation form	Commentary for user interpretation
L70-L72	`no_inv_resToheat`, `no_resToheat`, `no_heat_pump`	If `NoResToHeatInv=True`, force `heat_pump_cap` = `ResToHeat` = `heat_pump_out` = 0.	This switch does not relax heat equations; it removes one pathway (power-to-heat), so heat must be met by remaining technologies.
L239-L242	`ind_demand_meet`, `industry_demand_meet`	Fuel-share cap plus demand closure: $\sum_f \texttt{Fuel\_ind}_{f,t,y} + \texttt{H2tInd}_{t,y} = \texttt{Industry\_HT\_demand}_{t,y}$.	Industrial heat is always met; the optimization chooses fuel split within allowed shares and prices.
L247	`Power_to_h2`	Electrolyzer output divided by efficiency equals electric input for hydrogen production.	Electricity consumed by electrolysis is explicitly linked to hydrogen output and cannot be hidden elsewhere.
L255-L264	`h2_storage_*`	Hydrogen SOC recursion with upper/lower/terminal bounds.	Hydrogen can shift energy across hours, but bounded inventory prevents unrealistic unlimited shifting.
L284-L300	`ev_storage_*`	EV SOC recursion with transport withdrawal and charging efficiency; charge/discharge power limited by availability.	EV flexibility is conditional on parking/availability profile, not a free battery.
L305-L323	`stat_storage_*`	Stationary battery SOC recursion and investment-dependent SOC cap.	New storage capacity expands feasible SOC envelope over years.
L382 or L363	`meet_demand` or `meet_demand_export`	Hourly electric injections = withdrawals + coupling loads + slack.	This is the central adequacy equation; most infeasibilities propagate here when inputs are inconsistent.
L401	`ceep_demand`	Annual $\sum_t \texttt{ceep}_{t,y}$ bounded by `ceep_parameter` times annual electric needs.	Reliability slack exists but is policy-limited; tighter cap can make scenarios infeasible.
L414-L443	`heat_pump_*`, `heat_pump_soc_*`	COP-based electricity-to-heat relation and thermal SOC dynamics for heat pumps.	Clarifies why `heat_pump_ind` and `heat_pump_out` differ: one charges thermal storage and the other serves demand-side release.

Code reference	Constraint/object in code	Simplified equation form	Commentary for user interpretation
L467-L500	CHP/boiler/heat demand constraints	CHP extraction coupling, boiler caps, heat storage balance, heat and cooling demand closure.	Thermal demand is met through multiple parallel pathways; this block defines how those pathways interact hour-by-hour.
L515-L553	HDAM/HPHS storage constraints	Reservoir SOC bounds, inflow-driven recursion, generation/spill feasibility, terminal condition.	Prevents end-horizon depletion and enforces physically feasible hydro trajectories.
L560-L567	ramping and imports	Inter-hour ramp limits and import smoothing (±20% step) with NTC cap.	Controls operational realism and avoids abrupt imports or generator swings.
L571, L587-L602	`rps`, `CO2_emissions_*`	Renewable share minimum and annual emissions cap decomposition.	Portfolio and carbon policy are enforced after dispatch and sector-coupling interactions are accounted for.
L608-L612	`ramp_up/down_cost`	Auxiliary ramp cost variables lower-bound absolute generation changes (split sign-wise).	Adds economic penalty for fast dispatch variation while preserving linearity.
L652-L673	`objective`	Z = sum of all discounted OPEX/CAPEX/policy terms listed in the objective.	Confirms exactly which terms are monetized; useful when diagnosing unexpected investment or dispatch choices.

C.2 How to Read This Traceability Appendix

This appendix intentionally avoids reproducing long raw code listings. Instead, it provides a compact interpretation layer between the implemented model and the mathematical logic described earlier in the manual. The intended reading order is:

1. use Sections 4 and 5 to understand the model blocks conceptually;
2. use Appendix C.1 to identify how representative implemented constraints correspond to those blocks; and
3. use Appendix D to verify that the required input mappings and identifiers are consistent enough for those constraints to be meaningful.

D Appendix D: Complete Data Matching and Consistency Map

This appendix expands the core matching guidance from Section 3 into a more exhaustive audit map. The purpose is to distinguish between:

1. **hard joins in code**, where inconsistent keys directly break preprocessing or leave required values undefined;
2. **required schema fields**, where missing identifiers or column names make downstream construction impossible; and
3. **semantic consistency checks**, where the code may still run but the scenario interpretation becomes unreliable.

D.1 Hard Joins in Code

Source table / field	Must match with	Join keys	Join type in preprocessing	Consequence if inconsistent
hydro `genco.unit_name`	`scaled_inflows` columns	`year, period, unit_name` after melting inflow columns	explicit merge after wide-to-long transformation	Hydro inflow series are not assigned to the intended storage units; reservoir equations then operate with incomplete or missing inflow data.
non-dispatch `genco.unit_name`	`ncre_aval_factor` columns	`year, period, unit_name` after melting availability columns	explicit merge after wide-to-long transformation	Renewable availability is not linked to the intended unit; generation limits become wrong or missing for the affected assets.
`genco.fuel_type`	`fuel_cost` columns	`year, period, fuel_type`	left merge of long fuel-price table into `genco` time expansion	Variable generation cost cannot be built correctly; in the current preprocessing logic, missing values trigger a hard stop for affected units.
`thermal_tech.fuel_type`	`fuel_cost` columns	`year, period, fuel_type`	left merge into thermal technology expansion	Boiler and thermal-heat variable costs become undefined or inconsistent with the intended fuel assumptions.
`Industry.Fuel_source`	industry labels + `fuel_cost` columns	`Fuel_source`, then `year, period, Fuel_source`	merge of industrial-fuel metadata with long fuel-switching table	Industrial fuel-switching costs and emissions factors cannot be assembled consistently; some fuels may silently drop from the switching block.
`tech_cost_years` technology columns	`flex_tech.unit_name`	`year, unit_name` after melting technology columns	explicit merge to create annual CAPEX trajectories	Flex technologies receive no annual cost trajectory, so investment terms for those technologies become undefined or mismatched.
`COP_HP` technology columns	heat-pump identifiers in `flex_tech`	`year, period, variable` where `variable` must correspond to the heat-pump identifier	explicit merge after wide-to-long transformation	Heat-pump COP values are not linked to the correct technology, which distorts electricity-to-heat conversion and reserve/flexibility logic.
`DistrictHeating.unit`	DH market columns and linked equipment names	`unit` plus melted market identifier	explicit melt and restructuring into DH-capacity dictionaries	District-heating heat pumps and boilers are not mapped to the intended market/unit pairing, causing incorrect initial capacities and feasible heat supply structure.
CHP-linked `heat_demand` columns	CHP `genco.unit_name`	`year, period, unit_name` after melting demand columns	explicit merge-like alignment through long-format demand construction	CHP heat balance is attached to the wrong market label or not attached at all, invalidating thermal demand satisfaction.
cooling-demand market columns	cooling-service identifiers in the HP block	`year, period, unit_name`-style market naming after melt	long-format alignment into cooling demand dictionaries	Cooling demand may remain disconnected from heat-pump cooling variables, so the model solves a different cooling problem than intended.
`CO2_price.year`	industrial fuel-switching rows	`year`	left merge on annual table	Carbon-price signals are not propagated to industrial switching costs, causing incorrect cost comparison across fuels.
`import_export.period`	model `periods` set	`period`	direct alignment by period index	Import capacity and export trajectories become incomplete relative to the dispatch horizon.
`ev_transp_load.year, period`	model `years` and `periods`	`year, period`	direct alignment into EV availability and transport-load dictionaries	EV charging/discharging feasibility is built on an incomplete or shifted time grid.

D.2 Required Schema Columns and Identifier Fields

File / sheet	Required identifiers or columns	Category	Why they are structurally required
`genco_data_*.xml`	`unit_name`, `technology`, `fuel_type`, `cap_mw`, `efficiency`, `cost_no_fuel`, `cap_inv_cost`, `CO2Intensity`	master plant schema	This is the master table for generation assets. The model partitions units by technology, attaches time series by `unit_name`, and derives dispatch, cost, emissions, reserve, and lifetime behavior from these fields.
`genco_data_*.xml`	`RampUpRate`, `RampDownRate`, `RampingCost`	operational realism	These columns parameterize ramp constraints and ramping penalties in `Build_model.py`. Missing values weaken or invalidate operational flexibility representation.
`genco_data_*.xml`	`PrimaryReserve`, `SecondaryReserve`, `Stab_factor`	reserve eligibility	These fields determine reserve participation and reserve-down scaling for generation units. Missing or inconsistent values distort reserve feasibility.
`genco_data_*.xml`	`CHPType`, `CHPPowerToHeat`, `CHPPowerLossFactor`, `CHPMaxHeat`, `STOCapacity`, `STOSelfDischarge`	CHP and heat coupling	CHP heat-production limits and thermal storage dynamics depend on these identifiers. If a CHP unit exists without the matching CHP fields, the heat block becomes underdefined.
`genco_data_*.xml`	`decom_start_existing_cap`, `decom_start_new`, `life_time`, `final_life_cap`, `max_inv_period`	long-term planning	These fields feed decommissioning and investment-envelope logic. Missing values break the time evolution of effective capacity.
`flex_tech` sheet	`unit_name`, `Tech_type`, `efficiency`, `cap_mw`, `max_cap`, `var_cost`	flex technology master schema	These identify each non-genco flexibility asset and define its technical and economic envelope.
`flex_tech` sheet	`decom_start_new`, `life_time`, `final_life_cap`, `heat_storage_cap`	long-term flex evolution	These support investment, degradation/survival logic, and heat-storage coupling for heat pumps and storage technologies.
`flex_tech` sheet	`PrimaryReserve`, `SecondaryReserve`	reserve eligibility	These fields identify which flex technologies are allowed to provide reserve services.
`thermal_tech` sheet	`unit_name`, `fuel_type`, `cap_mw`, `max_cap`	thermal heat schema	These columns define boiler technologies and link them to fuel-price and emissions information.
`tech_cost_years` sheet	`year` plus one column per flex technology	annual CAPEX trajectories	The sheet is melted into (`year`, `unit_name`) cost rows. Missing technology columns mean the corresponding flex asset has no investment-cost path.
`COP_HP` sheet	`year`, `period` plus one column per heat-pump technology	hourly COP trajectories	The heat-pump electricity-to-heat relation is indexed on both year and period. The time grid must therefore exist explicitly.
`DistrictHeating` sheet	`unit` plus market-specific columns	DH market mapping	This sheet maps district-heating technologies to market labels and initial capacities used in the thermal block.
`Industry` sheet	`Fuel_source`, `Industry_HT`, `CO2_factor`	industrial switching schema	These columns define candidate industrial fuels, the industrial high-temperature demand context, and the fuel-level emissions factors.
`scaled_inflows`	`year`, `period` plus one column per hydro storage unit	time series input	Hydro inflow assignment depends on a complete time grid and exact unit-name matches after melt.
`ncre_aval_factor`	`year`, `period` plus one column per renewable unit	time series input	Renewable generation profiles are created from this file; mismatched columns leave units without availability factors.
`fuel_cost`	`year`, `period` plus one column per fuel	time series input	All fuel-based variable costs depend on this table after wide-to-long conversion.
`heat_demand`	`year`, `period`, CHP market columns, `general_demand`, `Industry_HT`	demand schema	The heat block expects both district-heating and general-demand labels; the industrial demand logic also expects the industrial high-temperature label to exist.
`cooling_demand`	`year`, `period`, `general_cooling` and any other cooling-market columns	demand schema	Cooling balances are built from these labels and must exist with the same time grid as the heat block.
`import_export`	`period`, `Import_capacity`, `total_exports`	interconnection schema	Import limits and exports are consumed directly by the electricity-balance logic.
`ev_transp_load`	`year`, `period`, `ev_availability`, `ev_transp_load`	EV time series schema	The EV block requires both grid-availability and transport-withdrawal series on the full model time grid.

D.3 Semantic Consistency Checks That May Not Trigger Immediate Errors

Consistency check	Files / objects involved	Why the run may still complete	Why the result can still be wrong
Technology taxonomy alignment	`genco.technology`, `flex_tech.Tech_type`, manual scenario switches in `main.py`	Labels can be syntactically present even if they are not used consistently across modules.	Units may be assigned to the wrong model subset, so constraints are applied to the wrong assets or omitted entirely.
Fuel taxonomy alignment	`genco.fuel_type`, `thermal_tech.fuel_type`, `Industry.Fuel_source`, fuel-cost columns	Generic strings can match partially or be interpreted inconsistently without an immediate parser failure.	Electricity, heat, and industry can end up using conceptually different fuels that appear similar by name, corrupting cost and emissions comparisons.
Year-grid consistency	`main.py` year lists and all annual or hourly input files	The code can process files with overlapping but not identical year domains.	Scenario trajectories, decommissioning vectors, CAPEX paths, and policy series can become misaligned across years.
Period-grid consistency	all hourly/time-slice files	A file may load even if some periods are missing or ordered differently.	Demand, inflow, availability, COP, transport load, and import limits may refer to different hourly structures.
District-heating market semantics	`DistrictHeating`, `heat_demand`, CHP and boiler market labels	Labels can exist in both places but represent different aggregation logic.	Heat is allocated to the wrong district-heating market, so feasible thermal service is misrepresented.
Cooling market semantics	`cooling_demand`, heat-pump cooling variables	Cooling labels may be present without a hard-coded failure.	Cooling demand can be solved against the wrong service bucket or omitted from the intended technology pathway.
Industrial high-temperature demand semantics	`heat_demand`, `Industry` sheet, industrial fuel-switching logic	The model can build the industrial switching block if identifiers exist, even if the demand interpretation is inconsistent.	Hydrogen substitution and industrial fuel switching may be benchmarked against the wrong thermal demand level.
Reserve switch coherence	`main.py` switches, reserve flags in `genco` and `flex_tech`	A reserve module can activate even when eligibility flags are poorly calibrated.	Reserve shortages or artificial reserve abundance can appear because participating units were not tagged consistently.
EV parameter interpretation	`main.py` EV/V2G assumptions and `ev_transp_load` time series	Both sources can be valid syntactically.	The aggregate battery size, power limits, and transport withdrawals may correspond to incompatible fleet assumptions.
Currency convention alignment	user manual convention, scenario comments in `main.py`, input cost files	Legacy comments do not stop execution.	Users may compare or calibrate monetary inputs inconsistently if some annotations still suggest USD while the model documentation assumes EUR.
Unit scaling conventions	MW vs. MWh-per-hour interpretations across demand, storage, and inflow files	Numerically similar scales can pass through the pipeline.	Scenario interpretation becomes wrong if one file is treated as hourly power while another is implicitly energy-per-step.

D.4 Recommended Pre-Run Review Logic

Before launching a new scenario, the user should verify the following in order:

1. **Identifier audit:** every required `unit_name`, `fuel_type`, market label, and technology label appears exactly as expected across all linked files.
2. **Time-grid audit:** all hourly inputs share the same `year` and `period` structure.
3. **Semantic audit:** market labels such as `general_demand`, `general_cooling`, and `Industry_HT` are interpreted consistently across all relevant files.
4. **Scenario audit:** annual vectors in `main.py` use the same horizon and ordering as the data files they are intended to parameterize.